



UNIVERSIDAD DE EXTREMADURA

CENTRO UNIVERSITARIO DE MÉRIDA

**GRADO EN INGENIERÍA INFORMÁTICA EN TECNOLOGÍAS DE LA
INFORMACIÓN.**

TRABAJO FIN DE GRADO

**Aplicación Multiplataforma para la Planificación Adaptativa de
Entrenamiento y Dieta**

Autora: LAURA REINOSA SÁNCHEZ

Mérida, Septiembre 2026



UNIVERSIDAD DE EXTREMADURA

CENTRO UNIVERSITARIO DE MÉRIDA

**GRADO EN INGENIERÍA INFORMÁTICA EN TECNOLOGÍAS DE LA
INFORMACIÓN.**

TRABAJO FIN DE GRADO

**Aplicación Multiplataforma para la Planificación Adaptativa de
Entrenamiento y Dieta**

Autora: LAURA REINOSA SÁNCHEZ

Fdo.: Firmado por REINOSA SANCHEZ
LAURA - ***9336** el día
01/09/2026 con un
certificado emitido por AC
FNMT Usuarios

Tutor: HÉCTOR SÁNCHEZ SANTAMARÍA

Fdo.: Firmado digitalmente
por SANCHEZ
SANTAMARIA HECTOR -
44678111J
Fecha: 2026.09.01
08:38:37 +02'00'

Agradecimientos

En primer lugar, quiero expresar mi agradecimiento a mi madre y a mi hermana, por nunca dejarme sola y apoyarme en cada etapa. Vuestro amor, vuestra confianza en mí y la seguridad de saber que siempre puedo contar con vosotras han sido mi mayor impulso durante todo este camino. Saber que pase lo que pase, siempre os tendré a mi lado es, sin duda, una de las mayores suertes de mi vida.

A mis mejores amigas, Paula y Marta, por haber estado siempre ahí, por ser mi apoyo incondicional, mi hogar y mi refugio. Por no haber dudado nunca de mí, por apostar siempre por cada una de mis metas y por ser esa familia que he tenido la suerte de poder elegir. Con vosotras he aprendido que todo pasa por algo y que, mientras ese «algo» me encuentre a vuestro lado, siempre me sentiré protegida. Una parte de todo lo que soy y de todo lo que consiga siempre será también vuestra.

A mi profesor y tutor de este TFG, por su orientación, dedicación y apoyo durante todo el desarrollo de este proyecto. Quiero agradecerle especialmente todo lo aprendido en la asignatura Desarrollo de Aplicaciones para Dispositivos Móviles, ya que los conocimientos adquiridos en ella han sido fundamentales para la realización de este trabajo. Gracias por su disponibilidad, por resolver mis dudas y por sus consejos a lo largo del desarrollo del TFG.

Resumen

En los últimos años han aparecido muchas aplicaciones relacionadas con el deporte, la alimentación y la salud. Aun así, la mayoría se centran solo en una de estas áreas u ofrecen planes muy generales que no son adaptados a los usuarios.

Por ello se ha desarrollado FitVive, una aplicación móvil compatible con Android e iOS que busca abarcar en una misma aplicación el entrenamiento, la alimentación y el seguimiento del progreso. La idea es que cada usuario pueda introducir sus datos, marcar sus objetivos y recibir planes y recomendaciones personalizados.

Entre las funciones principales de la aplicación están la creación de rutinas de entrenamiento personalizadas, planes de alimentación ajustados al perfil del usuario, el seguimiento de su evolución y recordatorios para ayudarle a mantener la constancia.

Para desarrollar la aplicación se ha utilizado Kotlin Multiplatform junto con Compose Multiplatform, lo que permite compartir gran parte del código entre Android e iOS. Además, se ha empleado Firebase Firestore para almacenar la información y varias APIs externas relacionadas con el ámbito del fitness y la nutrición.

En definitiva, FitVive pretende ofrecer una herramienta práctica y sencilla que ayude a organizar tanto el entrenamiento como la alimentación, facilitando el seguimiento de los progresos y haciendo que alcanzar los objetivos personales resulte más cómodo.

Abstract

In recent years, many applications related to sports, nutrition, and health have emerged. However, most of them focus on only one of these areas or offer very general plans that are not adapted to users' needs.

For this reason, FitVive has been developed as a mobile application compatible with Android and iOS that aims to combine training, nutrition, and progress tracking within a single application. The idea is to allow each user to enter their personal information, set their goals, and receive personalized plans and recommendations.

The main features of the application include the creation of personalized workout routines, meal plans adapted to the user's profile, progress tracking, and reminders to help users stay consistent.

The application has been developed using Kotlin Multiplatform together with Compose Multiplatform, which allows a large part of the code to be shared between Android and iOS. In addition, Firebase Firestore has been used to store information, along with several external APIs related to fitness and nutrition.

Overall, FitVive aims to provide a simple and practical tool that helps users organize both their training and nutrition, making it easier to track their progress and achieve their personal goals.

ÍNDICE GENERAL

Resumen.....	2
Abstract	3
1 Introducción y objetivos	7
1.1 Contexto y planteamiento del problema.....	8
1.2 Objetivos del proyecto.....	9
1.2.1 Objetivo principal	9
1.2.2 Objetivos específicos	9
1.3 Estructura del documento	9
2 Estado del arte y tecnologías	11
2.1 Aplicaciones actuales de entrenamiento	11
2.1.1 MyFitnessPal.....	11
2.1.2 Fitbod	12
2.1.3 Freeletics	13
2.1.4 Nike training club.....	15
2.1.5 Comparativa de las aplicaciones estudiadas	16
2.2 Tecnologías utilizadas	16
2.2.1 Alternativas para el desarrollo de aplicaciones	16
2.2.2 Kotlin Multiplatform	17
2.2.3 Compose Multiplatform	18
2.2.4 Firebase	19
2.2.5 Ktor.....	19
2.2.6 Gemini API	20
2.2.7 Wger API	20
2.2.8 Open Food Facts	20
3 Análisis del problema y requisitos	22
3.1 Análisis del problema	22
3.2 Alcance del sistema	22
3.3 Perfil del usuario y escenarios de uso.....	23
3.4 Requisitos funcionales	23
3.5 Requisitos no funcionales	24
3.6 Casos de uso.....	25
3.7 Viabilidad.....	35
3.7.1 Viabilidad técnica	35
3.7.2 Viabilidad económica	35
3.7.3 Viabilidad temporal	36
4 Metodología.....	38
4.1 Metodología y desarrollo	38
4.2 Planificación	39
5 Diseño e implementación.....	41
5.1 Diseño del sistema.....	41
5.1.1 Arquitectura de la aplicación.....	41
5.1.2 Organización de los componentes	42

5.1.3	Flujo general de funcionamiento	44
5.1.4	Integración multiplataforma.....	46
5.2	Diseño de la interfaz	47
5.2.1	Criterios de diseño	47
5.2.2	Organización de la navegación	47
5.2.3	Diseño de la interfaz del usuario.....	48
5.3	Modelo y persistencia de datos.....	55
5.3.1	Modelo de datos.....	56
5.3.2	Persistencia local	57
5.3.3	Sincronización.....	57
5.4	Comunicación con servicios externos	58
5.4.1	Servicio de ejercicios	58
5.4.2	Asistente virtual.....	59
5.4.3	Servicio de alimentación	59
5.5	Implementación de las funcionalidades principales	60
5.5.1	Gestión de usuarios.....	60
5.5.2	Planes de entrenamiento	61
5.5.3	Planificación de la dieta	61
5.5.4	Registro de progreso	62
6	Resultados y validación.....	65
6.1	Vistas definitivas de la aplicación	65
6.1.1	Pantalla de bienvenida.....	65
6.1.2	Registro e inicio de sesión	66
6.1.3	Configuración inicial.....	67
6.1.4	Pantalla principal	68
6.1.5	Plan de entrenamiento.....	69
6.1.6	Planificación de la dieta	72
6.1.7	Seguimiento del progreso	74
6.1.8	Asistente de IA.....	75
6.1.9	Perfil y configuración	76
6.2	Pruebas realizadas	78
6.2.1	Pruebas funcionales	78
6.2.2	Pruebas multiplataforma	79
6.3	Evaluación	79
6.3.1	Cumplimiento de los objetivos	80
6.3.2	Valoración del resultado	80
6.4	Limitaciones	80
7	Conclusiones y trabajos futuros	82
7.1	Conclusiones.....	82
7.2	Trabajos futuros	82
8	Bibliografía	84
9	Anexos	86
9.1	Manual de usuario	86
9.2	Manual de programador.....	89
9.3	Configuración de APIs	90
9.4	Código relevante	91

ÍNDICE DE TABLAS

Tabla 1. Caso de uso Registrarse. Fuente: Elaboración propia.	27
Tabla 2. Caso de uso Iniciar sesión. Fuente: Elaboración propia.	28
Tabla 3. Caso de uso Configurar perfil inicial. Fuente: Elaboración propia.	29
Tabla 4. Caso de uso Consultar plan de entrenamiento y registrar una sesión de entrenamiento. Fuente: Elaboración propia.	30
Tabla 5. Caso de uso Consultar plan de dieta y registrar el estado de las comidas. Fuente: Elaboración propia.	31
Tabla 6. Caso de uso Consultar progreso y gestionar los registros de peso. Fuente: Elaboración propia.	32
Tabla 7. Caso de uso Consultar y editar el perfil. Fuente: Elaboración propia.	33
Tabla 8. Caso de uso Consultar al asistente de inteligencia artificial. Fuente: Elaboración propia.	34
Tabla 9. Costes asociados a las tecnologías, herramientas e infraestructura utilizada durante el desarrollo de FitVive. Fuente: Elaboración propia.	35
Tabla 10. Distribución estimada a las horas dedicadas a cada fase del proyecto. Fuente: Elaboración propia.	36
Tabla 11. Planificación del proyecto. Fuente: Elaboración propia.	39
Tabla 12. Cronograma de planificación y desarrollo de FitVive. Fuente: Elaboración propia.	40
Tabla 13. Resumen de las pruebas funcionales realizadas. Fuente: Elaboración propia.	79
Tabla 14. Estructura del proyecto. Fuente: Elaboración propia.	89
Tabla 15. Tecnologías utilizadas. Fuente: Elaboración propia.	89

ÍNDICE DE FIGURAS

Figura 1. Interfaz de la aplicación MyFitnessPal.	12
Figura 2. Interfaz de la aplicación Fitbod.	13
Figura 3. Interfaz de la aplicación Freeletics.	15
Figura 4. Interfaz de la aplicación Nike Training Club.	16
Figura 5. Esquema general del funcionamiento de Kotlin Multiplatform	19
Figura 6. Flujo de comunicación entre FitVive y Gemini API	21
Figura 7. Diagrama de casos de uso.	27
Figura 8. Fases del desarrollo de FitVive	39
Figura 9. Arquitectura general de FitVive.	43
Figura 10. Organización código compartido (commonMain).	44
Figura 11. Flujo general de funcionamiento de FitVive.	47
Figura 12. Organización del código compartido y específico de cada plataforma en FitVive.	48
Figura 13. Mapa de navegación de las principales pantallas de FitVive.	50
Figura 14. Logotipo de FitVive.	51
Figura 15. Diseño previo de la pantalla de bienvenida de FitVive.	51
Figura 16. Diseño previo del proceso de configuración inicial de FitVive.	52
Figura 17. Diseño previo de la pantalla principal de FitVive realizado en Figma.....	53
Figura 18. Diseño previo de la pantalla de entrenamientos de FitVive.	54
Figura 19. Diseño previo de la pantalla de Dieta de FitVive.	55
Figura 20. Diseño previo de la pantalla Perfil de FitVive.	57
Figura 21. Diagrama Entidad-Relación.	58
Figura 22. Registro de peso y mensaje del análisis informativo.	65
Figura 23. Pantalla de progreso con la gráfica de evolución de peso.	66
Figura 24. Pantalla de bienvenida de FitVive en Android e iOS.	68
Figura 25. Pantallas de inicio de sesión y registro.	69
Figura 26. Proceso de configuración inicial de FitVive.	70
Figura 27. Pantalla principal de FitVive en Android e iOS.	71
Figura 28. Pantalla principal de entrenamientos.	72
Figura 29. Detalle de un entrenamiento.	73
Figura 30. Pantalla principal de la planificación de la dieta.	74
Figura 31. Detalle de la planificación de la dieta.	75
Figura 32. Pantalla principal del seguimiento del progreso.	76
Figura 33. Registro de un nuevo peso.	77
Figura 34. Pantalla del Asistente IA.	78
Figura 35. Pantalla principal del perfil.	79
Figura 36. Pantalla de edición del perfil.	79
Figura 37. Pantalla de configuración de notificaciones.	80
Figura 38. Código correspondiente al registro de un usuario.	94
Figura 39. Código correspondiente a la realización de un entrenamiento.	95
Figura 40. Código correspondiente a la obtención de información nutricional mediante Open Food Facts.	96
Figura 41. Código correspondiente al envío de consultas al asistente basado en inteligencia artificial.	97
Figura 42. Código correspondiente a la programación de notificaciones diarias	98

1 Introducción y objetivos

En este capítulo se introduce el proyecto FitVive, explicando el contexto y la motivación que dieron lugar a su desarrollo. También se presentan el problema que se pretende resolver, los objetivos del proyecto y la estructura general del presente documento.

1.1 Contexto y planteamiento del problema

En los últimos años, el interés por la actividad física y la adopción de hábitos de vida saludables ha experimentado un crecimiento considerable. Según la Organización Mundial de la Salud (OMS), la realización de la actividad física aporta numerosos beneficios para la salud física y mental, además de contribuir a prevenir diversas enfermedades crónicas [1]. Sin embargo, muchas personas, especialmente los jóvenes, continúan recurriendo a las redes sociales para obtener información sobre nutrición y entrenamiento. En estos medios es frecuente encontrar contenido generado por personas sin la formación adecuada o información que prioriza intereses comerciales antes que el rigor científico, lo que puede llevar a seguir recomendaciones poco fiables e incluso perjudiciales para la salud.

Como consecuencia, han surgido numerosas aplicaciones móviles orientadas al entrenamiento físico y la alimentación, con el objetivo de ayudar a los usuarios a mejorar su salud y alcanzar diferentes metas deportivas. Sin embargo, la mayoría se especializan únicamente en el entrenamiento o en la alimentación. En aquellos casos en los que ambas funcionalidades se encuentran integradas, los planes suelen presentar un nivel de personalización limitado y no se adaptan a la evolución del usuario. Esta falta de personalización puede dificultar que cada persona alcance sus objetivos de forma eficaz.

Para dar respuesta a estas limitaciones, se desarrolla FitVive, una aplicación móvil multiplataforma diseñada para ofrecer una experiencia personalizada tanto en el ámbito del entrenamiento como de la nutrición. La aplicación genera planes adaptados a las características y objetivos de cada usuario, permitiendo seleccionar aspectos como el número de días de entrenamiento, el objetivo físico (ganancia de masa muscular, pérdida de grasa o mantenimiento) y otros parámetros personales.

Además, FitVive incorpora funcionalidades adicionales como el seguimiento del progreso del usuario, recordatorios periódicos para favorecer la constancia y un asistente conversacional basado en inteligencia artificial que permite resolver dudas relacionadas con el entrenamiento y la alimentación. De esta forma, la aplicación busca acompañar al usuario durante todo su proceso, adaptándose a su evolución y favoreciendo la adquisición de hábitos saludables.

Finalmente, FitVive está dirigida a un público amplio, ya que su nivel de personalización permite que pueda ser utilizada tanto por personas que se inician en el ejercicio físico como por usuarios con mayor experiencia que desean planificar y controlar su entrenamiento y alimentación desde una única plataforma.

1.2 Objetivos del proyecto

1.2.1 Objetivo principal

El objetivo principal de este Trabajo de Fin de Grado es diseñar y desarrollar una aplicación móvil multiplataforma que permita la generación y gestión de planes personalizados de entrenamiento y nutrición, permitiendo que cualquier persona pueda acceder a ellos de forma sencilla. Además, se proporciona información basada en datos fiables y contrastados, evitando la generación de planes genéricos y ofreciendo una planificación personalizada y segura.

1.2.2 Objetivos específicos

Para alcanzar el objetivo principal, se plantean los siguientes objetivos específicos:

- Registrar información detallada de cada usuario.
- Generar planes de entrenamientos adaptados al perfil y preferencias de cada usuario.
- Generar planes nutricionales personalizados, según las necesidades y objetivos de cada usuario.
- Registrar el progreso del usuario.
- Mostrar estadísticas que faciliten la evolución del usuario.
- Enviar notificaciones diarias y semanales, para recordar entrenamientos, comidas y otras actividades relacionadas con el seguimiento del usuario.
- Desarrollar una aplicación compatible con el sistema operativo de Android y de iOS.
- Diseñar una interfaz intuitiva y sencilla que facilite el uso de la aplicación para cualquier tipo de usuario.

Integrar un sistema basado en inteligencia artificial que permita resolver dudas relacionadas con el entrenamiento y la nutrición.

1.3 Estructura del documento

La presente memoria se estructura en nueve capítulos que describen las distintas fases del desarrollo del proyecto.:

1. **Introducción y objetivos.** Se presenta el contexto del proyecto, el problema que se pretende resolver y los objetivos planteados.
2. **Estado del arte y tecnologías.** Se analizan algunas aplicaciones similares a la planteada y se describen las tecnologías y servicios utilizados para el desarrollo de FitVive.
3. **Análisis del problema y requisitos.** Se explica el problema que da origen al proyecto, el perfil del usuario, los requisitos de la aplicación, los casos de uso y el estudio de viabilidad.
4. **Metodología.** Se describe la forma en la que se ha organizado y desarrollado el proyecto.
5. **Diseño e implementación.** Se explica el diseño general de la aplicación, la arquitectura, el diseño de la interfaz, el modelo de datos, la integración con los servicios externos y el desarrollo de las principales funcionalidades.
6. **Resultados y validación.** Se presentan las funcionalidades desarrolladas, las pruebas realizadas, la evaluación del prototipo y las principales limitaciones.

7. **Conclusiones y trabajos futuros.** Se presentan las principales conclusiones derivadas del desarrollo del proyecto, así como las posibles mejoras, ampliaciones y nuevas funcionalidades que podrían abordarse como líneas de trabajo futuro.
8. **Bibliografía.** Se recopilan las fuentes de información utilizadas durante la realización del trabajo.
9. **Anexos.** Se incluyen documentos complementarios, como el manual de usuario, el manual del programador, la configuración de las APIs y el código fuente más relevante.

2 Estado del arte y tecnologías

En este capítulo se presentan las aplicaciones disponibles en el mercado de apps móviles de entrenamiento y nutrición. Además, se describen las principales tecnologías utilizadas durante la implementación de FitVive y se explica brevemente el motivo de su elección.

2.1 Aplicaciones actuales de entrenamiento

En la actualidad existe una amplia variedad de aplicaciones destinadas a teléfonos inteligentes relacionadas con la dieta, el entrenamiento y la actividad física. La App Store española cuenta con más de veinte páginas de aplicaciones dentro de la categoría <<Salud y forma física>> [2]. Muchas de ellas ofrecen funcionalidades avanzadas que permiten a los usuarios planificar entrenamientos, controlar la alimentación o registrar su evolución, como Strava, MyFitnessPal, Nike Training Club o Fitify. Sin embargo, estas aplicaciones pueden presentar ciertas limitaciones relacionadas con la personalización o el acceso a todas sus funcionalidades.

A continuación, se analizarán las aplicaciones más conocidas y utilizadas en la actualidad.

2.1.1 MyFitnessPal

MyFitnessPal es una aplicación móvil centrada en el seguimiento de la nutrición, diseñada para ayudar a mejorar los hábitos saludables y alcanzar los objetivos relacionados con la salud y la condición física [3].

Es una aplicación muy completa, ya que entre sus principales funciones se encuentran las siguientes [3]:

- Contar calorías, registrando los alimentos consumidos.
- Realizar un seguimiento de los macronutrientes de cada comida.
- Escanear el código de barras de los alimentos.
- Registrar los ejercicios realizados.
- Establece objetivos personales.
- Consultar el progreso relacionado con la alimentación.

MyFitnessPal destaca por su amplia base de datos de alimentos y por ofrecer una interfaz sencilla e intuitiva, lo que facilita el registro diario de la alimentación y de la actividad física. Estas características permiten al usuario llevar un seguimiento detallado de su alimentación y de sus hábitos de entrenamiento.

Sin embargo, también presenta algunas desventajas. Su uso requiere que el usuario registre los alimentos y bebidas que consume, lo que puede resultar poco práctico a largo plazo. Aunque la aplicación ayuda a controlar las calorías y los macronutrientes, no garantiza que esta sea una alimentación equilibrada o adecuada para los objetivos del usuario. Por último, algunas funcionalidades solamente están disponibles con una suscripción Premium [3].

En la Figura 1 se muestra la interfaz principal de MyFitnessPal, donde pueden observarse algunas de las funcionalidades disponibles en la aplicación.

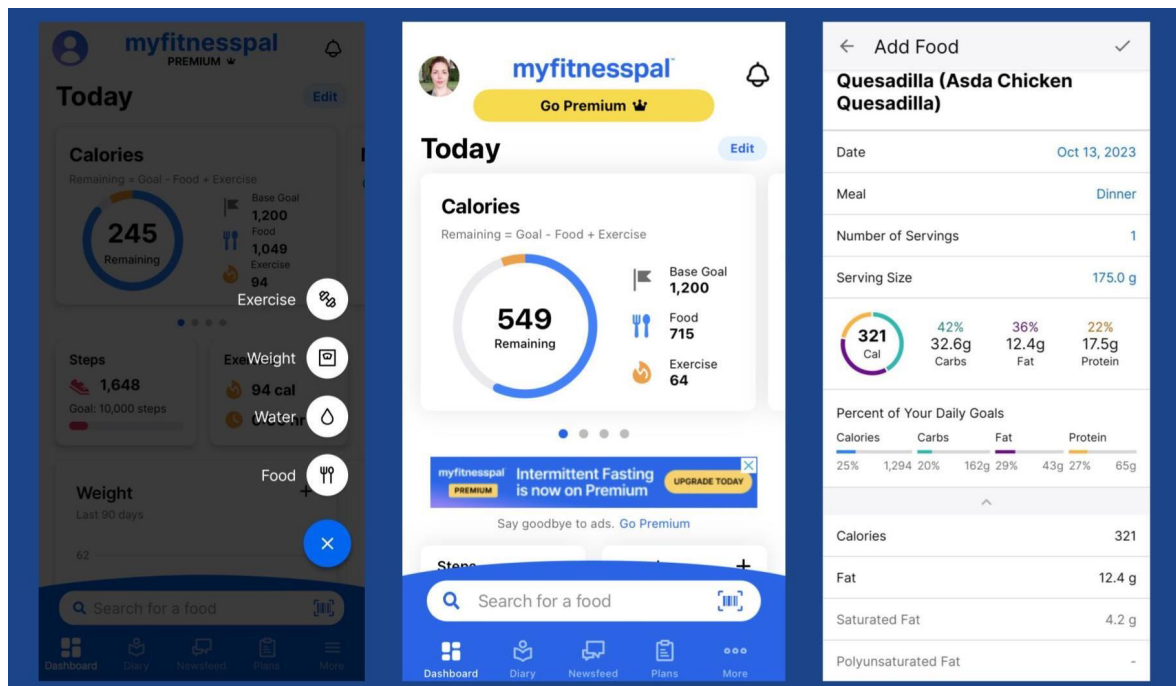


Figura 1. Interfaz de la aplicación MyFitnessPal. Fuente: [20].

2.1.2 Fitbod

Fitbod es una aplicación de entrenamiento personalizado que genera rutinas de forma automática en función de los objetivos, la experiencia y el material disponible de cada usuario. Su funcionamiento se asemeja al de un entrenador personal, ya que el usuario debe registrar el rendimiento en cada sesión, y la aplicación adapta los entrenamientos en función de sus necesidades y progreso [4].

Algunas de sus funcionalidades más importantes son [4]:

- Generación automática de rutinas.
- Seguimiento de recuperación muscular.
- Progresión automática de cargas en el entrenamiento.
- Registro de estadísticas en función de la evolución del usuario.

Fitbod destaca por ofrecer una experiencia muy completa centrada en el entrenamiento, ya que genera automáticamente rutinas personalizadas según los objetivos, la experiencia y el material disponible de cada usuario. Además, dispone de una amplia variedad de ejercicios y de una interfaz sencilla e intuitiva, lo que facilita su utilización tanto en el gimnasio como en casa.

Sin embargo, también presenta algunas limitaciones. La aplicación no sustituye completamente el asesoramiento de un entrenador personal y algunas de sus funcionalidades requieren una suscripción. Además, al estar centrada principalmente en

el entrenamiento, no ofrece el mismo nivel de seguimiento en otros aspectos relacionados con la alimentación.

En la Figura 2 se muestran algunas de las pantallas de Fitbod, donde pueden observarse las rutinas de entrenamiento y el seguimiento de los ejercicios.



Figura 2. Interfaz de la aplicación Fitbod. Fuente: [21].

2.1.3 Freeletics

Freeletics es una aplicación de entrenamiento basada en un entrenador con inteligencia artificial que crea planes personalizados para entrenar en casa, en el gimnasio o al aire libre. Está orientada a mejorar la condición física general. Su AI Coach adapta el entrenamiento en función del progreso del usuario y de la información proporcionada después de cada sesión [5].

Algunas de las funcionalidades más importantes son [5]:

- AI Coach personalizado, que adapta el entrenamiento en función del progreso y de la disponibilidad del usuario.
- Programas de entrenamiento orientados a diferentes objetivos, como la pérdida de grasa, la hipertrofia, la fuerza o la resistencia.
- Posibilidad de entrenar tanto con equipamiento como sin él, adaptando el entrenamiento al material disponible.
- Amplia variedad de ejercicios con demostraciones e instrucciones para su realización.
- Adaptación del entrenamiento según el progreso del usuario.

- Seguimiento del progreso.

Freeletics destaca por ofrecer una experiencia de entrenamiento completa y flexible, ya que permite realizar los entrenamientos tanto en casa como en el gimnasio o al aire libre. Además, permite trabajar diferentes capacidades físicas, como la fuerza, la resistencia, la movilidad, la flexibilidad y la capacidad cardiovascular.

Sin embargo, también presenta algunas limitaciones. Al tratarse de un sistema automatizado, la personalización no puede alcanzar el mismo nivel que el seguimiento realizado directamente por un profesional. Además, algunas de sus funcionalidades más avanzadas solo están disponibles mediante una suscripción de pago [5].

La Figura 3 muestra algunas de las pantallas de Freeletics, donde pueden observarse diferentes opciones relacionadas con los entrenamientos y su seguimiento.

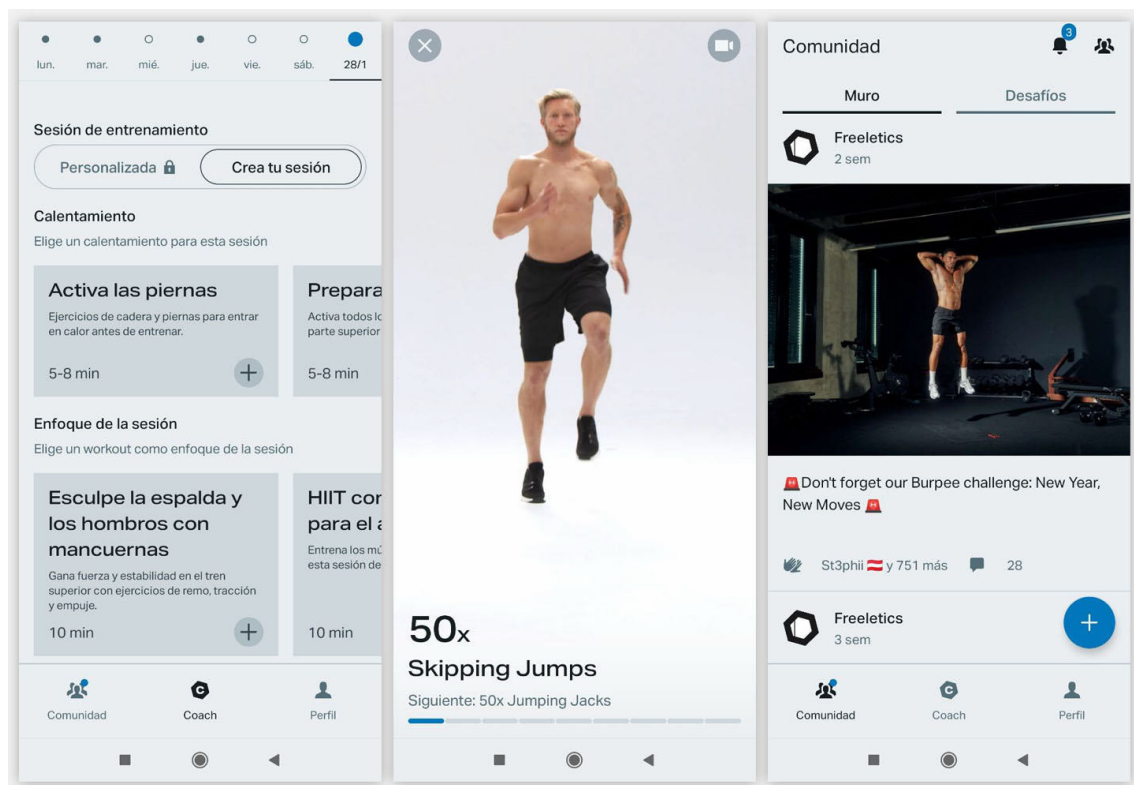


Figura 3. Interfaz de la aplicación Freeletics. Fuente [22]

2.1.4 Nike training club

Nike Training Club es una aplicación de entrenamiento desarrollada por Nike, diseñada para ayudar a usuarios de diferentes niveles a mejorar su condición física mediante entrenamientos guiados. La aplicación ofrece una amplia variedad de sesiones y programas de entrenamientos dirigidos por entrenadores de Nike [6].

Algunas de sus funcionalidades más importantes son [6]:

- Amplia variedad de entrenamientos orientados a diferentes niveles.
- Programas dirigidos a distintos objetivos, como ganar fuerza, mejorar la movilidad o desarrollar hábitos saludables.
- Videos guiados con explicaciones para facilitar la realización de los ejercicios.
- Seguimiento de la actividad y del progreso del usuario.
- Posibilidad de realizar diferentes tipos de entrenamientos, como fuerza, movilidad, yoga o cardio.

Nike Training Club destaca por ofrecer una amplia variedad de entrenamientos guiados y programas adaptados a diferentes objetivos. Además, incorpora vídeos con instrucciones claras para facilitar la correcta realización de los ejercicios, lo que hace que sea una aplicación accesible para usuarios con distintos niveles de experiencia.

No obstante, también presenta algunas limitaciones. El nivel de personalización es reducido, ya que la aplicación no adapta los entrenamientos en función del rendimiento del usuario después de cada sesión. Además, no genera una planificación automática, ya que es el propio usuario el que debe seleccionar el próximo entrenamiento.

La Figura 4 muestra algunas de las pantallas de Nike Training Club, donde pueden observarse diferentes entrenamientos y programas disponibles.

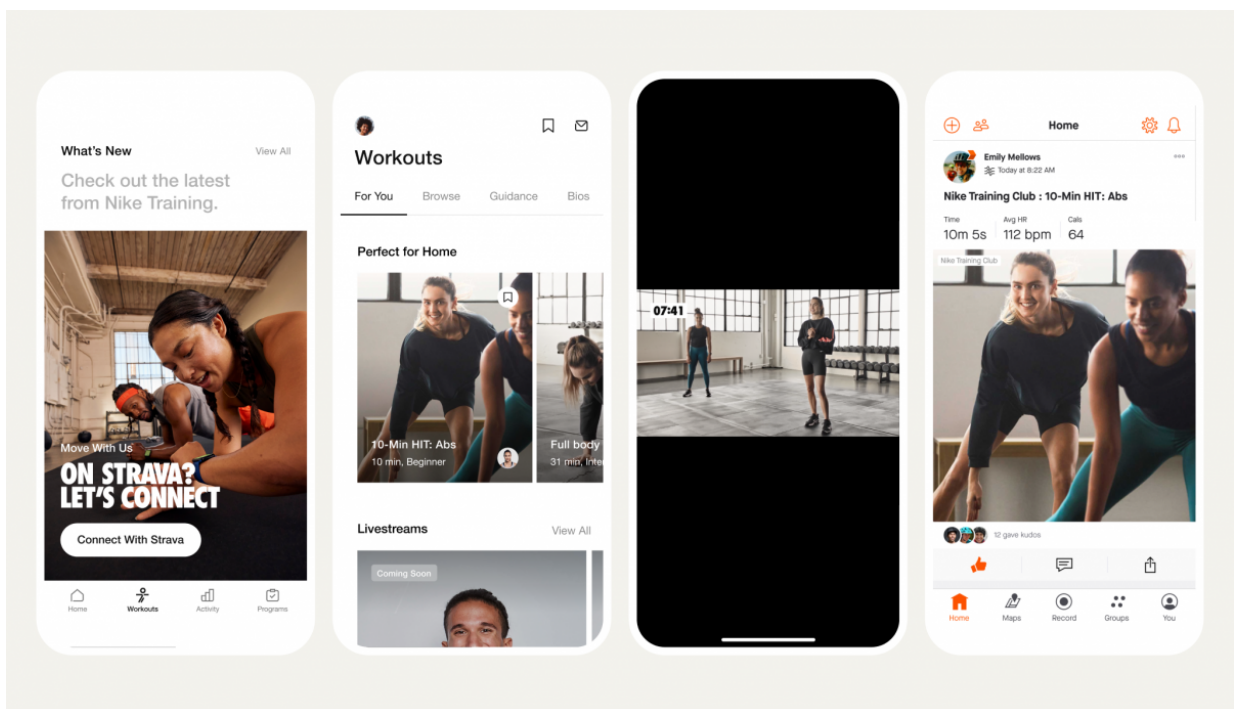


Figura 4. Interfaz de la aplicación Nike Training Club. Fuente: [23].

2.1.5 Comparativa de las aplicaciones estudiadas

Una vez analizadas las aplicaciones anteriores, se pueden observar diferentes características y enfoques entre ellas.

MyFitnessPal está orientada principalmente al seguimiento de la alimentación, mientras que Fitbod, Freeletics y Nike Training Club se centran en el entrenamiento físico.

Fitbod y Freeletics ofrecen un mayor nivel de personalización de los entrenamientos, mientras que Nike Training Club destaca por disponer de una amplia variedad de entrenamientos. Por otro lado, MyFitnessPal ofrece una gran variedad de funcionalidades relacionadas con la alimentación.

Frente a estas aplicaciones, FitVive reúne en una misma plataforma funcionalidades relacionadas tanto con el entrenamiento como con la alimentación. La aplicación genera planes adaptados a cada usuario en función de sus características y objetivos, permitiendo además realizar un seguimiento de su progreso. También incorpora recordatorios y un asistente basado en inteligencia artificial para resolver dudas relacionadas con el entrenamiento y la alimentación.

Por tanto, la principal diferencia de FitVive respecto a las aplicaciones estudiadas es la integración de estas funcionalidades en una única aplicación, evitando que el usuario tenga que utilizar varias aplicaciones para realizar el seguimiento de su entrenamiento y alimentación.

2.2 Tecnologías utilizadas

En este apartado se describen las principales tecnologías y herramientas utilizadas durante el desarrollo de la aplicación. Además, se explicará brevemente el funcionamiento de cada una de ellas y el motivo por el cual han sido seleccionadas para implementar las distintas funcionalidades de la aplicación.

2.2.1 Alternativas para el desarrollo de aplicaciones

Existen diferentes formas de desarrollar una aplicación móvil para los distintos sistemas operativos. Entre las principales opciones se encuentran el desarrollo nativo, el desarrollo web y el desarrollo multiplataforma.

El desarrollo nativo consiste en crear una aplicación específica para cada sistema operativo. Por ejemplo, para Android se pueden utilizar lenguajes como Kotlin, mientras que para iOS se utiliza principalmente Swift [7][8]. Este tipo de desarrollo permite aprovechar las funcionalidades propias de cada plataforma, aunque es necesario desarrollar y mantener un código diferente para ambas plataformas.

Por otro lado, el desarrollo web permite crear aplicaciones utilizando tecnologías web como HTML, CSS y JavaScript. Estas aplicaciones se utilizan principalmente desde un navegador y se puede acceder a ellas desde diferentes dispositivos. Sin embargo, presentan algunas limitaciones para acceder a determinadas funciones del dispositivo.

Otra alternativa es el desarrollo multiplataforma, que permite compartir gran parte del código de una aplicación entre los diferentes sistemas operativos, como Android e iOS. Para este tipo de desarrollo existen tecnologías como Flutter, React Native o Kotlin Multiplatform. De esta forma, no es necesario desarrollar toda la aplicación por separado para cada plataforma, lo que facilita su desarrollo y mantenimiento.

Para el desarrollo de FitVive se ha decidido utilizar Kotlin Multiplatform, ya que permite compartir gran parte del código entre Android e iOS y, al mismo tiempo, desarrollar de forma independiente aquellas funcionalidades que son específicas para cada plataforma [9].

2.2.2 Kotlin Multiplatform

Kotlin Multiplatform (KMP), desarrollado por JetBrains, es una tecnología diseñada para facilitar el desarrollo de aplicaciones multiplataformas. Su principal objetivo es permitir reutilizar una gran parte del código entre diferentes plataformas, utilizando un único lenguaje de programación: Kotlin. KMP adopta un enfoque híbrido, permitiendo reutilizar la lógica principal de la aplicación mientras se mantiene una interfaz de usuario (UI) nativa para cada plataforma [9].

En el código compartido es posible implementar la lógica de negocio de la aplicación, incluyendo las llamadas API, la obtención y sincronización de datos, la autenticación de usuarios, el almacenamiento de información y el manejo de errores. La mayor parte del funcionamiento de la aplicación puede desarrollarse una única vez y reutilizarse posteriormente tanto en Android como en iOS.

Esta forma de trabajo ofrece diferentes ventajas durante el desarrollo, como la reutilización del código entre plataformas, la reducción del tiempo de desarrollo y un mantenimiento más sencillo ya que se dispone de una única base de código. Además, permite mantener la misma lógica de funcionamiento en ambas plataformas, garantizando un comportamiento similar en ambas plataformas y reduciendo la aparición de errores [9].

Se ha elegido Kotlin Multiplatform para el desarrollo de la aplicación FitVive porque permite compartir la lógica de negocio entre Android e iOS, reduciendo el tiempo de desarrollo y facilitando el mantenimiento de la aplicación y garantizando un funcionamiento coherente en ambas plataformas.

La Figura 5 muestra de forma esquemática la organización general de Kotlin Multiplatform, diferenciando el código compartido y el código específico para cada plataforma.

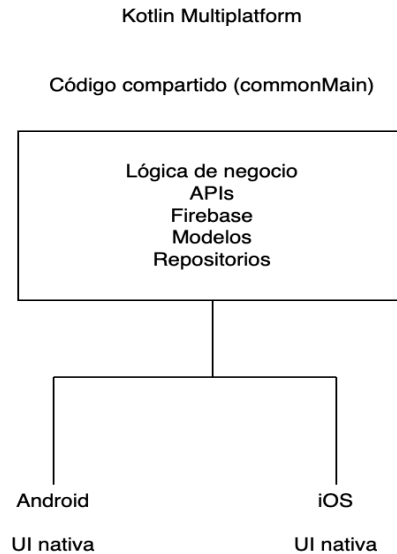


Figura 5. Esquema general del funcionamiento de Kotlin Multiplatform. Fuente: Elaboración propia.

2.2.3 Compose Multiplatform

Compose Multiplatform es un framework desarrollado por JetBrains y basado en Kotlin que permite crear interfaces de usuario para múltiples plataformas, como Android, iOS, escritorio y web, compartiendo gran parte del mismo código. Su objetivo es facilitar el desarrollo de aplicaciones multiplataforma mediante la reutilización de componentes visuales, reduciendo el tiempo de desarrollo y simplificando el mantenimiento de la interfaz [10].

Una de sus principales características es el uso de una interfaz declarativa, donde el desarrollador describe cómo debe mostrarse la interfaz, sin necesidad de indicar paso a paso cómo debe construirse. Además, permite crear componentes reutilizables que pueden utilizarse en distintas partes de la aplicación, actualiza automáticamente la interfaz cuando cambian los datos de la aplicación, evitando tener que modificar manualmente cada elemento visual. También se integra de forma natural con Kotlin Multiplatform, lo que permite compartir tanto la lógica de negocio como la interfaz entre varias plataformas. Cuando es necesario, ofrece la posibilidad de combinar el código compartido con implementaciones nativas.

Compose Multiplatform organiza la interfaz mediante componentes denominados composables. Cada composable representa una parte de la interfaz, como un botón, una tarjeta o un campo de texto, que puede reutilizarse en diferentes pantallas de la aplicación. Esto facilita la organización del código y hace que el desarrollo y el mantenimiento de la interfaz sean más sencillos.

En el desarrollo de FitVive, Compose Multiplatform se ha utilizado para implementar todas las pantallas de la aplicación, permitiendo reutilizar gran parte de la interfaz entre Android e iOS. Esto ha facilitado el desarrollo y ha permitido mantener el mismo diseño y funcionamiento en ambos sistemas operativos.

2.2.4 Firebase

Cloud Firestore es una base de datos NoSQL alojada en la nube que forma parte de Firebase. Los datos se organizan en colecciones y documentos, lo que permite estructurar los datos de forma flexible y realizar consultas de manera eficiente. Además, los cambios se sincronizan en tiempo real entre los dispositivos conectados, y la información se mantiene disponible incluso cuando la aplicación se encuentra sin conexión. Una vez recuperado el acceso a Internet, los datos se sincronizan automáticamente con la base de datos [11].

Existen diferentes alternativas para almacenar y gestionar los datos desde una aplicación móvil. Además de Firebase, se pueden utilizar otras opciones como Supabase, que ofrece servicios de base de datos y autenticación. Otra opción sería desarrollar un backend propio utilizando bases de datos como MySQL o PostgreSQL. Sin embargo, estas opciones tienen diferentes formas de configuración y mantenimiento.

Se ha decidido usar Cloud Firestore en el desarrollo de FitVive porque se necesitaba una base de datos escalable, fácil de integrar y capaz de compartir determinada información entre distintos dispositivos sin necesidad de desarrollar un backend propio. De esta manera, Firebase permite almacenar y sincronizar determinados datos entre dispositivos, simplificando el desarrollo de la aplicación. En FitVive, Cloud Firestore se utiliza para almacenar la información del perfil del usuario y sincronizar determinados datos, como las comidas completadas, los registros de peso y las estadísticas de progreso, principalmente en la versión de Android.

2.2.5 Ktor

Ktor es un framework desarrollado por JetBrains que permite realizar comunicaciones mediante peticiones HTTP utilizando Kotlin. Es compatible con Kotlin Multiplatform, por lo que permite reutilizar la misma lógica de comunicación con servicios externos tanto en Android como en iOS, evitando duplicar el código y aprovechando la lógica compartida del proyecto [12].

En FitVive, se ha utilizado Ktor para conectar la aplicación con las APIs externas empleadas. Gracias a ello ha sido posible centralizar la comunicación con estos servicios dentro del código compartido, evitando implementar una solución diferente para cada plataforma, facilitando el desarrollo y mantenimiento del proyecto.

2.2.6 Gemini API

Gemini API es una interfaz de programación proporcionada por Google que permite integrar los modelos de inteligencia artificial generativa de Gemini en una aplicación. A través de esta API, una aplicación puede enviar información a los modelos de inteligencia artificial y recibir respuestas generadas, permitiendo añadir funcionalidades basadas en IA como generación de texto y análisis de información [13].

En FitVive, se ha utilizado Gemini API para implementar un asistente conversacional basado en inteligencia artificial. La aplicación envía las consultas realizadas por el usuario al modelo de Gemini mediante peticiones HTTP y recibe una respuesta generada automáticamente, que se mostrará en la aplicación para resolver las dudas relacionadas con el entrenamiento, nutrición o hábitos saludables que el usuario pudiera tener.

La Figura 6 muestra el flujo de comunicaciones entre FitVive y Gemini API, desde el envío de la consulta realizada por el usuario hasta la recepción de la respuesta generada por Gemini.

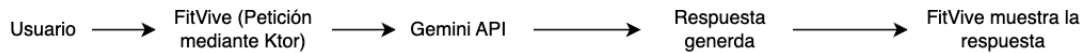


Figura 6. Flujo de comunicación entre Fitvive y Gemini API. Fuente: Elaboración propia.

2.2.7 Wger API

Wger API es una API REST de código abierto especializada en el ámbito del entrenamiento físico. Esta API proporciona una amplia base de datos de ejercicios organizados por categorías, grupos musculares, equipamiento y otros criterios. Además, muchos de los ejercicios incluyen imágenes ilustrativas que ayudan al usuario a comprender como debe realizarse cada movimiento correctamente [15].

Wger API se ha utilizado en FitVive para obtener los ejercicios que forman parte de los planes de entrenamiento personalizados. La aplicación realiza consultas a la API para obtener la información necesaria ya adaptar los ejercicios del grupo muscular correspondiente, incorporando además datos como el número de series, repeticiones, y tiempo de descanso. Gracias a ello ha sido posible disponer de una amplia variedad de ejercicios sin la necesidad de crear una base de datos propia.

2.2.8 Open Food Facts

Open Food Facts es una API REST de código abierto que proporciona una amplia base de datos sobre productos alimenticios. Dicha información incluye datos como el nombre del producto, los ingredientes, valores nutricionales y otros aspectos relacionados con la nutrición [16].

Se ha utilizado esta API en FitVive para obtener la información nutricional de los alimentos empleados para la generación de los planes de alimentación. La aplicación consulta la API para recuperar los alimentos necesarios según las necesidades del usuario. Posteriormente, FitVive aplica su propia lógica en la que se seleccionan los alimentos y se construye una dieta personalizada a cada usuario según los objetivos establecidos por este. Este enfoque ha permitido desarrollar

3 Análisis del problema y requisitos

En este capítulo se analizarán las necesidades que debe satisfacer FitVive para dar respuesta a los requisitos y expectativas de sus usuarios. Para ello, se analizarán las necesidades que debe cubrir la aplicación, el alcance del sistema, el perfil de usuario, los requisitos funcionales y no funcionales, los casos de uso y la viabilidad del proyecto. Todo ello establecerá la base para su posterior diseño e implementación.

3.1 Análisis del problema

Hoy en día, existen numerosas personas que desean cambiar su condición física, pero existen varios factores en contra que pueden impedir alcanzar sus objetivos. Por ejemplo, una falta de conocimientos para poder planificar una dieta o un entrenamiento, falta de tiempo, o limitaciones económicas para poder permitirse un entrenador personal. Algunas aplicaciones pueden llegar a alcanzar estos objetivos, pero la mayoría de estas aplicaciones generan planes poco personalizados a cada usuario, o son incompletas (solo existe una planificación de entrenamiento o de dieta, pero no ambas).

FitVive surge con el propósito de dar respuesta a estas necesidades, de tal manera que el usuario pueda introducir en la aplicación sus objetivos o necesidades. Con esta información, se podrá construir una dieta y un plan de entrenamiento personalizado, teniendo en cuenta el seguimiento del progreso del perfil registrado. Además, contará con un asistente conversacional para poder resolver cualquier duda que pueda surgir acerca del ámbito fitness.

A partir de estas necesidades se han definido una serie de requisitos funcionales y no funcionales que la aplicación debe cumplir para ofrecer una solución adaptada a los usuarios.

3.2 Alcance del sistema

El presente proyecto tiene como alcance el desarrollo de una aplicación multiplataforma orientada a la planificación personalizada del entrenamiento y la alimentación. Para ello, la aplicación permite la gestión del perfil del usuario, la generación de planes personalizados de entrenamiento y alimentación, el seguimiento del progreso, el registro de comidas y entrenamientos, el envío de notificaciones y la incorporación de un asistente conversacional basado en inteligencia artificial. Asimismo, la aplicación ha sido desarrollada para funcionar tanto en dispositivos Android como iOS.

No obstante, el sistema presenta una serie de limitaciones que quedan fuera del alcance del proyecto. FitVive no pretende sustituir el trabajo de un nutricionista o de un entrenador personal, no realiza diagnósticos médicos ni genera planes específicos para personas con patologías. Tampoco permite la interacción entre usuarios, ni la sincronización con relojes inteligentes.

3.3 Perfil del usuario y escenarios de uso

FitVive está dirigida a personas interesadas en mejorar su condición física mediante una planificación personalizada del entrenamiento y la alimentación. La aplicación está concebida tanto para usuarios que se inician en el ámbito del fitness como para aquellos que ya cuentan con experiencia previa, sin que sea necesario disponer de conocimientos avanzados sobre entrenamiento o nutrición.

La aplicación permite adaptar la planificación a las características, necesidades y objetivos de cada usuario. Entre estos objetivos se encuentran la pérdida de grasa, la ganancia de masa muscular o el mantenimiento de la condición física. De este modo, FitVive pretende dar respuesta a diferentes perfiles de usuario, teniendo en cuenta tanto su nivel de experiencia como sus objetivos personales.

A partir de este perfil general, pueden identificarse distintos escenarios representativos que permiten concretar las principales situaciones en las que los usuarios pueden recurrir a FitVive:

Usuario principiante. El usuario desea mejorar su condición física, pero no dispone de los conocimientos necesarios para organizar adecuadamente su entrenamiento y alimentación. FitVive le proporciona una planificación adaptada a sus características y objetivos y le permite realizar un seguimiento de su evolución.

Usuario que entrena habitualmente. El usuario cuenta con experiencia previa y busca organizar sus rutinas y controlar su progreso. FitVive le permite consultar sus entrenamientos, registrar su evolución y adaptar la planificación cuando cambian sus objetivos o necesidades.

Usuario que necesita resolver dudas. Durante el seguimiento de su planificación pueden surgir dudas relacionadas con el entrenamiento, la alimentación o determinados hábitos. Para atender estas consultas, FitVive incorpora un asistente conversacional integrado en la aplicación.

3.4 Requisitos funcionales

A continuación, se definen las funcionalidades necesarias que la aplicación debe ofrecer para satisfacer las necesidades del usuario:

1. **Gestión del perfil del usuario.** La aplicación deberá proporcionar un sistema de registro y de inicio de sesión donde el usuario pueda registrar y gestionar sus datos personales. Además, deberá permitir al usuario configurar sus objetivos, preferencias y características personales, información que servirá como base para personalizar el funcionamiento de la aplicación.

2. **Personalización de la planificación.** Una vez se ha completado el perfil del usuario, la aplicación generará un plan de entrenamiento y de dieta adaptados al perfil del usuario, adaptada según los datos, características y objetivos aportados. Si el usuario modifica sus datos, esta planificación será modificada también.
3. **Gestión del entrenamiento.** Una vez se haya generado el plan de entrenamiento, el usuario podrá consultar la rutina propuesta, y marcar los ejercicios realizados como completados. Se registrarán pesos y repeticiones, y esto será guardado para el seguimiento del progreso.
4. **Gestión de la alimentación.** Se generará un plan de alimentación semanal, en la que se podrá ver las comidas generadas diarias, los ingredientes de estas comidas, y los macros de cada comida. Además, el usuario deberá marcar como completado cuando este complete cualquier comida.
5. **Seguimiento del progreso.** Deberá integrarse un sistema que facilite a los usuarios realizar el seguimiento del progreso semanal o diario de cada uno. Se puede consultar la evolución del peso, las estadísticas, y los entrenamientos completados.
6. **Asistente conversacional.** Se implementará un sistema de asistente conversacional donde se podrán resolver las dudas que puedan surgir relacionadas con el ámbito del fitness.
7. **Sistema de notificaciones.** La aplicación consta de un sistema de notificaciones diarias y semanales, permitiendo al usuario configurar la hora en la que la notificación sea visible. Habrá notificaciones que salgan dentro de la aplicación, o notificaciones que salten al dispositivo móvil fuera de la aplicación. Esto ayudará a que el usuario este pendiente a registrar las comidas o entrenamientos propuestos, y que pueda llevar una constancia.

3.5 Requisitos no funcionales

A continuación, se describen los principales requisitos no funcionales que debe cumplir el proyecto. Esto hace referencia a aspectos técnicos y de calidad, que son fundamentales para garantizar el correcto funcionamiento de la aplicación.

1. **Usabilidad.** La aplicación deberá ofrecer una interfaz sencilla e intuitiva, que sea fácil de utilizar para todo tipo de usuarios, ya que está pensada para usuarios con y sin experiencia. La navegación deberá ser clara y facilitar el acceso a las principales funcionalidades de la aplicación.
2. **Rendimiento.** La aplicación deberá responder de forma fluida durante la utilización, minimizando el tiempo de carga en la navegación y en la generación de planes personalizados. Además, la comunicación con la base de datos local y con las APIs externas deberá realizarse de manera eficiente para ofrecer una experiencia satisfactoria.

3. **Compatibilidad.** FitVive ha sido desarrollada mediante Kotlin Multiplatform, por lo que deberá garantizar un funcionamiento correcto tanto en Android como en iOS, manteniendo un comportamiento similar en ambas plataformas.
4. **Seguridad y privacidad.** La aplicación protegerá la información de los usuarios, manteniendo su privacidad, garantizando que únicamente el usuario autenticado pueda acceder a su información personal.
5. **Persistencia y fiabilidad.** La aplicación deberá garantizar el acceso a los datos almacenados localmente incluso cuando no exista conexión a Internet. Para ello, FitVive utiliza una base de datos local mediante SQLDelight, lo que permite consultar información como el perfil del usuario, el progreso, los entrenamientos registrados o la planificación disponible. No obstante, aquellas funcionalidades que dependan de servicios externos, como el asistente conversacional o la consulta de información mediante APIs, requerirán conexión a Internet.
6. **Mantenibilidad.** La aplicación cuenta con una arquitectura organizada y modular que facilita la incorporación de futuras mejoras, la corrección de errores y el mantenimiento del código sin afectar al funcionamiento del sistema.
7. **Escalabilidad.** Se deberá permitir incorporar nuevas funcionalidades sin la necesidad de modificar la estructura de la aplicación. Esto facilitará la integración futura de nuevas funcionalidades, manteniendo un rendimiento adecuado.

3.6 Casos de uso

En este apartado se presentan los principales casos de uso de FitVive, con el objetivo de representar las distintas funcionalidades que ofrece la aplicación y las interacciones que el usuario puede realizar con el sistema.

3.7.1. Introducción a los casos de uso

Los casos de uso permiten representar las principales funcionalidades que ofrece FitVive desde el punto de vista del usuario. A través de ellos, se describen las distintas acciones que pueden realizarse dentro de la aplicación, facilitando la comprensión de las interacciones entre el usuario y el sistema. De esta forma, es posible identificar de manera clara las funcionalidades disponibles y la relación existente entre el usuario y la aplicación.

3.7.2 Diagrama de casos de uso

El diagrama de casos de uso mostrado a continuación representa las principales funcionalidades implementadas en FitVive y las interacciones que el usuario puede realizar con la aplicación. A través de él se ofrece una visión general del funcionamiento del sistema y de las relaciones existentes entre los distintos casos de uso.

Para la elaboración del diagrama se ha utilizado draw.io, mediante el cual se representa el actor principal del sistema, el usuario, junto con los diferentes casos de uso asociados a las funcionalidades implementadas en la aplicación [18].

A continuación, la Figura 7 muestra el diagrama de los casos de uso de FitVive.

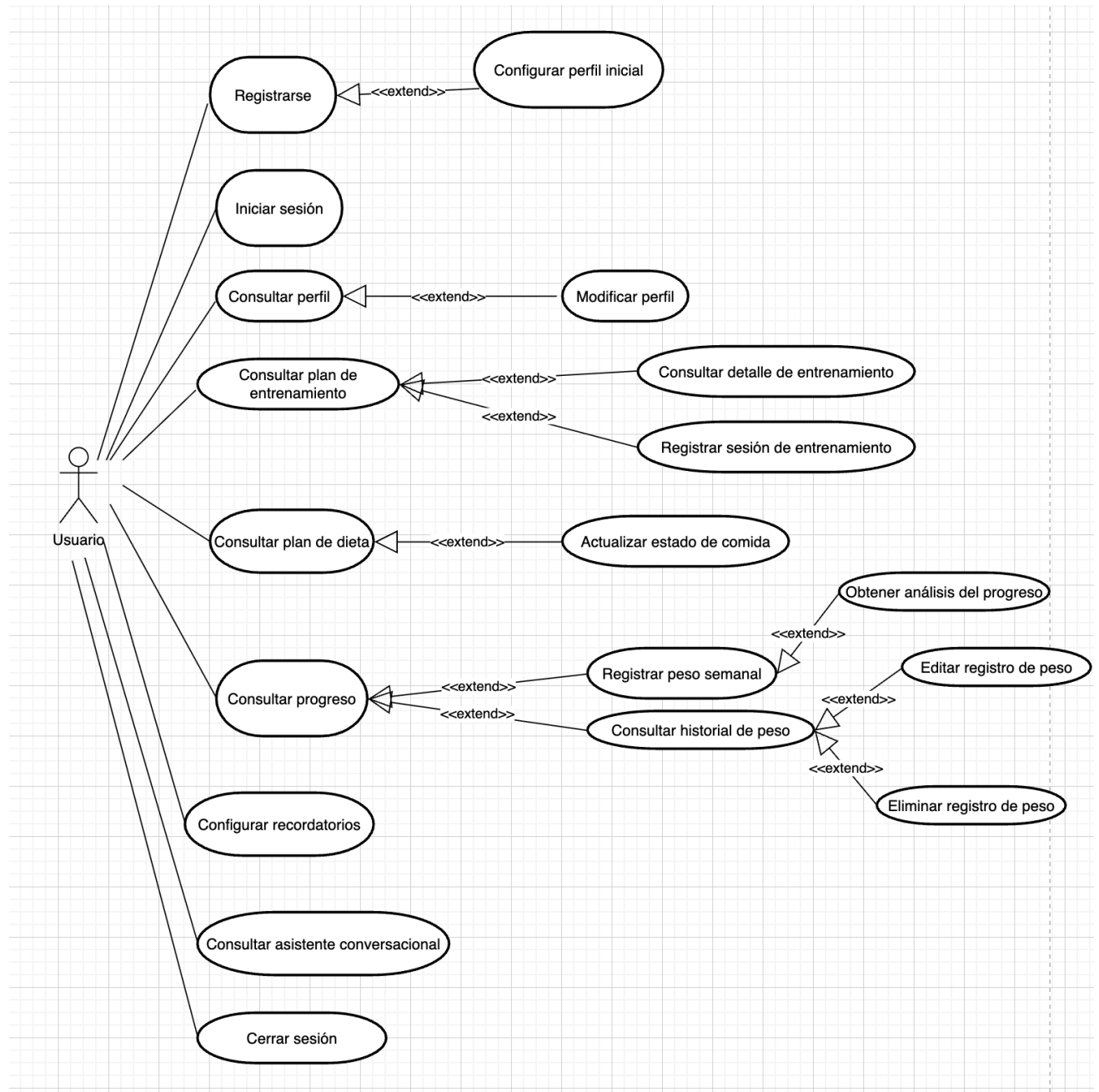


Figura 7. Diagrama de casos de uso. Fuente: Elaboración propia.

3.7.3 Descripción de los principales casos de uso

En este apartado se describen los principales casos de uso principales implementados en FitVive. Cada uno de ellos representa una de las funcionalidades principales de la aplicación y está relacionado con los requisitos funcionales definidos previamente.

ID caso de uso	CU-01
Nombre caso de uso:	Registrarse
Actores:	Usuario sin cuenta local registrada
Descripción	Permite al usuario crear una nueva cuenta en FitVive, mediante el nombre, el correo electrónico, la contraseña y su confirmación. Antes de guardar los datos, la aplicación valida la información introducida y comprueba que no exista otra cuenta registrada en el dispositivo.
Evento que lo genera	El usuario accede a la pantalla de registro, completa el formulario y pulsa Registrarse.
Precondiciones	La aplicación se encuentra en la pantalla de registro y no existe una cuenta local previamente registrada.
Postcondiciones	La cuenta queda almacenada localmente, se muestra un mensaje de confirmación y el usuario puede continuar a la pantalla de inicio de sesión.
Flujo normal	1. El usuario introduce el nombre, correo, contraseña y confirmación. 2. Pulsa Registrarse. 3. El sistema valida la información introducida 4. El sistema comprueba que no exista una cuenta local. 5. Almacena los datos. 6. Muestra la confirmación del registro. 7. El usuario pulsa Continuar y accede al inicio de sesión.
Flujo alternativo	1. Si el nombre está vacío, se muestra el error correspondiente. 2. Si el correo no tiene un formato válido, se informa al usuario. 3. Si la contraseña tiene menos de seis caracteres, se muestra un error. 4. Si las contraseñas no coinciden, no se completa el registro. 5. Si ya existe una cuenta local, se impide el registro y se muestra un aviso.

Tabla 1. Caso de uso Registrarse. Fuente: Elaboración propia.

ID caso de uso	CU-02
Nombre caso de uso:	Iniciar sesión
Actores:	Usuario registrado
Descripción	Permite al usuario acceder a FitVive introduciendo el correo electrónico y la contraseña registrados previamente en la aplicación.
Evento que lo genera	El usuario accede a la pantalla de inicio de sesión, introduce sus credenciales y pulsa el botón “ENTRAR”.
Precondiciones	- Existe un usuario registrado en la aplicación. - El usuario dispone de unas credenciales válidas.
Postcondiciones	- Si las credenciales son correctas, el usuario accede a la aplicación - Si el perfil aún no ha sido configurado completamente, el sistema muestra el proceso de configuración inicial. - Si las credenciales son incorrectas, el usuario permanece en la pantalla de inicio de sesión.
Flujo normal	1. El usuario introduce su correo electrónico. 2. El usuario introduce su contraseña. 3. Pulsa el botón ENTRAR. 4. La aplicación valida el formato del correo electrónico. 5. La aplicación comprueba que las credenciales coinciden con las almacenadas. 6. Si la validación es correcta, recupera los datos del usuario. 7. Si el perfil ya está configurado, accede a la pantalla principal. 8. En caso contrario, accede al proceso de configuración inicial.
Flujo alternativo	FA-01. Correo electrónico no válido: 1. El usuario introduce un correo electrónico con un formato incorrecto. 2. La aplicación muestra el mensaje “Introduce un correo válido”. 3. No se comprueban las credenciales. 4. El usuario permanece en la pantalla de inicio de sesión.
	FA-02. Credenciales incorrectas. 1. El usuario introduce un correo electrónico o una contraseña que no coinciden con los datos almacenados. 2. La aplicación muestra el mensaje “Credenciales incorrectas”. 3. No se inicia sesión. 4. El usuario permanece en la pantalla de inicio de sesión.
	FA-03. Acceder al registro. 1. El usuario selecciona la opción “¿No tienes cuenta? Regístrate”. 2. La aplicación muestra la pantalla de registro. 3. La aplicación cancela el proceso de inicio de sesión y muestra la pantalla de registro
	FA-04. Regresar a la pantalla inicial. 1. El usuario pulsa la opción de retroceso desde la pantalla de inicio de sesión. 2. La aplicación vuelve a la pantalla principal.

Tabla 2. Caso de uso Iniciar sesión. Fuente: Elaboración propia.

ID caso de uso	CU-03
Nombre caso de uso:	Configurar perfil inicial.
Actores:	Usuario registrado
Descripción	Permite al usuario completar la configuración inicial de su perfil introduciendo sus datos físicos, su objetivo, los días de entrenamiento semanales y el grupo muscular al que desea dar su prioridad.
Evento que lo genera	El usuario inicia sesión correctamente y su perfil todavía no tiene configurados los días de entrenamiento.
Precondiciones	- Existe un usuario registrado en la aplicación. - El usuario ha iniciado sesión correctamente. - El perfil inicial todavía no ha sido completado.
Postcondiciones	- Los datos introducidos quedan almacenados localmente. - El perfil queda configurado con el objetivo, los días de entrenamiento y el grupo muscular prioritario seleccionados. - El usuario accede a la pantalla principal de la aplicación.
Flujo normal	1. La aplicación muestra la primera pantalla de configuración. 2. El usuario introduce su sexo, peso, altura y fecha de nacimiento. 3. Pulsa SIGUIENTE. 4. La aplicación valida los datos introducidos. 5. El usuario selecciona su objetivo físico. 6. Selecciona el número de días de entrenamiento semanales. 7. Selecciona el grupo muscular al que desea dar prioridad. 8. Pulsa COMENZAR. 9. La aplicación guarda la configuración del perfil. 10. El usuario accede a la pantalla principal.
Flujo alternativo	FA-01. Sexo no indicado. 1. El usuario deja vacío el campo de sexo. 2. La aplicación muestra el mensaje “Indica tu sexo”. 3. El usuario permanece en la primera pantalla.
	FA-02. Peso no válido 1. El peso no es numérico o no se encuentra entre 30 y 300 kg. 2. La aplicación muestra “Introduce un peso válido (30-300 kg)”. 3. El usuario no puede avanzar.
	FA-03. Altura no válida 1. La altura no es numérica o no se encuentra entre 100 y 250 cm. 2. La aplicación muestra “Introduce una altura válida (100-250 cm)”. 3. El usuario no puede avanzar.
	FA-04. Fecha con formato incorrecto. 1. La fecha no cumple el formato DD/MM/AAAA. 2. La aplicación muestra “Usa el formato DD/MM/AAAA”. 3. El usuario permanece en la primera pantalla
	FA-05. Opciones no seleccionadas. 1. El usuario no selecciona el objetivo, los días de entrenamiento o el grupo muscular prioritario. 2. El botón para continuar permanece deshabilitado. 3. El usuario debe realizar una selección para avanzar.

Tabla 3. Caso de uso Configurar perfil inicial. Fuente: Elaboración propia.

ID caso de uso	CU-04
Nombre caso de uso:	Consultar plan de entrenamiento y registrar una sesión de entrenamiento.
Actores:	Usuario con el perfil inicial configurado.
Descripción	Permite al usuario consultar el plan de entrenamiento personalizado generado y registrar la realización de una sesión de entrenamiento.
Evento que lo genera	El usuario accede al apartado “Entrenamientos” desde el menú principal de la aplicación.
Precondiciones	- El usuario ha iniciado sesión. - El perfil inicial ha sido configurado correctamente. - Existe un plan de entrenamiento asociado al usuario.
Postcondiciones	- La sesión de entrenamiento queda registrada. - La aplicación almacena la información correspondiente al entrenamiento realizado. - El usuario vuelve al listado de entrenamientos.
Flujo normal	1. El usuario accede al apartado “Entrenamientos”. 2. La aplicación muestra el plan de entrenamiento personalizado. 3. El usuario selecciona uno de los días disponibles. 4. La aplicación muestra los ejercicios correspondientes a la sesión. 5. El usuario consulta la información de los ejercicios. 6. Introduce el peso y las repeticiones realizadas en cada serie. 7. Marca las series completadas. 8. Pulsa el botón “FINALIZAR ENTRENAMIENTO”. 9. La aplicación registra la sesión realizada. 10. El usuario vuelve al listado de entrenamientos.
Flujo alternativo	FA-01. No se pueden obtener los ejercicios desde el servicio externo. 1. La aplicación no obtiene ejercicios desde el servicio externo. 2. Se muestra automáticamente la rutina interna de la aplicación. 3. El usuario puede continuar con el entrenamiento.
	FA-02. Ejercicio sin imagen 1. Un ejercicio no dispone de imagen. 2. La aplicación muestra el mensaje “Sin imagen disponible”. 3. El resto de la información del ejercicio permanece disponible.
	FA-03. Finalizar el entrenamiento sin completar todas las series. 1. El usuario pulsa “FINALIZAR ENTRENAMIENTO” sin completar todas las series. 2. La aplicación registra igualmente la sesión. 3. El usuario vuelve al listado de entrenamientos.
	FA-04. Abandonar la sesión 1. El usuario sale de la pantalla antes de finalizar el entrenamiento. 2. La sesión no queda registrada. 3. Los datos introducidos se pierden.

Tabla 4. Caso de uso Consultar plan de entrenamiento y registrar una sesión de entrenamiento. Fuente: Elaboración propia.

ID caso de uso	CU-05
Nombre caso de uso:	Consultar plan de dieta y registrar el estado de las comidas.
Actores:	Usuario con el perfil inicial configurado.
Descripción	Permite al usuario consultar un plan de alimentación personalizado según el objetivo configurado en su perfil y marcar las comidas realizadas para llevar un seguimiento diario.
Evento que lo genera	El usuario accede al apartado “Dieta” desde la pantalla principal o desde el menú inferior.
Precondiciones	- El usuario ha iniciado sesión. - El perfil inicial ha sido configurado correctamente. - El usuario tiene un objetivo establecido.
Postcondiciones	- El estado de las comidas completadas queda almacenado localmente. - Al volver a acceder al mismo día, las comidas marcadas continúan apareciendo como completadas. - La aplicación regresa a la pantalla principal de la dieta.
Flujo normal	1. El usuario accede al apartado “Dieta”. 2. La aplicación muestra los días de la semana disponibles. 3. El usuario selecciona un día. 4. La aplicación genera el plan alimentario correspondiente según el objetivo configurado. 5. Se muestran las comidas del día con su información nutricional. 6. El usuario marca una o varias comidas como completadas. 7. La aplicación actualiza el estado de las comidas y lo almacena localmente. 8. El usuario pulsa “FINALIZAR DÍA”. 9. La aplicación vuelve a la pantalla principal de la dieta.
Flujo alternativo	FA-01. Desmarcar una comida. 1. El usuario pulsa sobre una comida previamente marcada como completada. 2. La aplicación elimina la marca de completada. 3. El nuevo estado queda almacenado localmente.
	FA-02. No existen comidas completadas. 1. El usuario accede a un día sin comidas registradas. 2. Todas las comidas aparecen como pendientes. 3. El usuario puede comenzar a marcarlas como completadas.
	FA-03. Finalizar el día sin completar todas las comidas. 1. El usuario pulsa “FINALIZAR DÍA” sin haber marcado todas las comidas. 2. La aplicación no muestra ningún mensaje de advertencia. 3. Las comidas ya marcadas permanecen almacenadas y las restantes continúan pendientes.

Tabla 5. Caso de uso Consultar plan de dieta y registrar el estado de las comidas. Fuente: Elaboración propia.

ID caso de uso	CU-06
Nombre caso de uso:	Consultar el progreso y gestionar los registros de peso.
Actores:	Usuario con el perfil inicial configurado.
Descripción	Permite al usuario consultar su evolución física, registrar nuevos pesos, visualizar el historial de registros y gestionar la información almacenada mediante su edición o eliminación.
Evento que lo genera	El usuario accede al apartado “Mi progreso” desde la pantalla principal.
Precondiciones	- Existe un usuario registrado en la aplicación. - El usuario ha iniciado sesión correctamente. - El usuario dispone de un perfil activo
Postcondiciones	- El nuevo registro de peso queda almacenado. - El historial de peso se actualiza. - El usuario puede consultar la evolución de su progreso.
Flujo normal	1. El usuario accede al apartado “Mi progreso”. 2. La aplicación muestra el resumen del progreso del usuario. 3. Se visualizan los indicadores principales y el historial de peso disponible. 4. El usuario pulsa “+REGISTRAR PESO”. 5. Introduce el nuevo peso. 6. Pulsa “GUARDAR REGISTRO”. 7. La aplicación valida el peso introducido. 8. El nuevo registro se almacena. 9. La aplicación genera un análisis del progreso. 10. El usuario cierra el análisis y vuelve a la pantalla de progreso.
Flujo alternativo	FA-01. No existen registros de peso. 1. El usuario accede al apartado de progreso. 2. La aplicación indica que todavía no existen registros. 3. El usuario puede crear el primer registro de peso.
	FA-02. Peso no válido 1. El usuario introduce un peso fuera del rango permitido o con un formato incorrecto. 2. La aplicación muestra “Introduce un peso válido (30-300 kg)”. 3. El usuario permanece en la pantalla hasta corregir el dato.
	FA-03. Editar un registro. 1. El usuario selecciona un registro del historial. 2. Modifica el peso o las notas. 3. La aplicación actualiza la información almacenada.
	FA-04. Eliminar un registro. 1. El usuario selecciona la opción de eliminar. 2. La aplicación solicita confirmación. 3. El usuario confirma la eliminación. 4. El registro desaparece del historial.
	FA-05. Cancelar el registro 1. El usuario pulsa “CANCELAR” antes de guardar. 2. La aplicación vuelve a la pantalla de progreso. 3. No se almacena ningún registro.

Tabla 6. Caso de uso Consultar progreso y gestionar los registros de peso. Fuente: Elaboración propia.

ID caso de uso	CU-07
Nombre caso de uso:	Consultar y editar el perfil.
Actores:	Usuario con el perfil inicial configurado.
Descripción	Permite al usuario consultar la información de su perfil y modificar determinados datos personales y deportivos, como el peso, la altura, el objetivo, los días de entrenamiento y el grupo muscular prioritario.
Evento que lo genera	El usuario accede al apartado “Perfil” desde el menú inferior de la aplicación.
Precondiciones	- El usuario ha iniciado sesión. - El perfil inicial ha sido configurado correctamente.
Postcondiciones	- Los cambios realizados en el perfil quedan almacenados. - La información mostrada en el perfil se actualiza. - Si el usuario cancela la edición, no se modifica ningún dato.
Flujo normal	1. El usuario accede al apartado “Perfil”. 2. La aplicación muestra la información personal y deportiva del usuario. 3. El usuario pulsa “EDITAR PERFIL”. 4. Modifica el peso, la altura, el objetivo, los días de entrenamiento o el grupo muscular prioritario. 5. Pulsa “GUARDAR CAMBIOS”. 6. La aplicación valida los datos introducidos. 7. Los cambios se almacenan y el perfil se actualiza. 8. La aplicación vuelve a modo de consulta de perfil.
Flujo alternativo	FA-01. Peso no válido 1. El usuario introduce un peso fuera del rango permitido o con un formato incorrecto. 2. La aplicación muestra “Introduce un peso válido (30-300 kg)”. 3. El usuario permanece en la pantalla hasta corregir el dato.
	FA-03. Altura no válida 1. La altura no es numérica o no se encuentra entre 100 y 250 cm. 2. La aplicación muestra “Introduce una altura válida (100-250 cm)”. 3. El usuario no puede avanzar.
	FA-03. Cancelar la edición. 1. El usuario pulsa “CANCELAR”. 2. La aplicación descarta los cambios realizados. 3. Se muestran nuevamente los datos originales del perfil.
	FA-04. Cerrar sesión. 1. El usuario pulsa “CERRAR SESIÓN”. 2. La aplicación vuelve a la pantalla inicial. 3. El usuario deberá volver a acceder para utilizar la aplicación.

Tabla 7. Caso de uso Consultar y editar el perfil. Fuente: Elaboración propia.

ID caso de uso	CU-08
Nombre caso de uso:	Consultar al asistente de inteligencia artificial.
Actores:	Usuario con el perfil inicial configurado.
Descripción	Permite al usuario realizar consultas relacionadas con entrenamiento, nutrición y hábitos saludables mediante un asistente basado en inteligencia artificial.
Evento que lo genera	El usuario accede al apartado “Asistente IA” desde la pantalla principal.
Precondiciones	- El usuario ha iniciado sesión. - El perfil inicial ha sido configurado correctamente. - El dispositivo dispone de conexión a Internet.
Postcondiciones	- La conversación se muestra en la pantalla durante la sesión actual. - No se almacena el historial de mensajes al abandonar el chat. - No se modifica ningún dato del perfil del usuario.
Flujo normal	1. El usuario accede al apartado “Asistente IA”. 2. La aplicación muestra una conversación nueva con un mensaje de bienvenida. 3. El usuario escribe una consulta relacionada con entrenamiento o nutrición. 4. Pulsa el botón de envío. 5. La aplicación envía la consulta junto con parte de la información del perfil del usuario. 6. Mientras se procesa la petición, se muestra un indicador de carga. 7. El asistente devuelve una respuesta. 8. La respuesta se añade a la conversación. 9. El usuario puede continuar realizando nuevas consultas.
Flujo alternativo	FA-01. Mensaje vacío. 1. El usuario intenta enviar un mensaje vacío. 2. El botón de envío permanece deshabilitado. 3. No se realiza ninguna petición.
	FA-02. Error de conexión. 1. Se produce un error durante la comunicación con el servicio. 2. La aplicación muestra el mensaje “Error de conexión. Verifica tu internet e inténtalo de nuevo.” 3. El usuario puede volver a intentarlo.
	FA-03. Respuesta no válida. 1. El servicio no devuelve una respuesta válida. 2. La aplicación muestra el mensaje “No pude obtener una respuesta. Inténtalo de nuevo.” 3. El usuario puede realizar una nueva consulta.
	FA-04. Salir del asistente. 1. El usuario abandona la pantalla del asistente. 2. La conversación actual se pierde. 3. Al volver a acceder se inicia un nuevo chat con el mensaje de bienvenida.

Tabla 8. Caso de uso Consultar al asistente de inteligencia artificial. Fuente: Elaboración propia.

3.7 Viabilidad

En este apartado se analiza la viabilidad del proyecto desde las perspectivas técnica, económica y temporal, con el objetivo de determinar si su desarrollo es factible considerando las tecnologías y los recursos disponibles, los costes asociados y el tiempo previsto para su ejecución.

3.7.1 Viabilidad técnica

La viabilidad técnica del proyecto viene determinada por la utilización de tecnologías que permiten desarrollar una aplicación de forma eficiente.

La aplicación está desarrollada con Kotlin Multiplatform, lo que permite desarrollar una única aplicación para Android e iOS. Además, Compose Multiplatform facilita el desarrollo de una interfaz de usuario común para ambas plataformas.

SQLDelight permite almacenar información localmente [17]. Cloud Firestore complementa el almacenamiento local mediante la sincronización parcial de determinados datos en dispositivos Android cuando existe conexión a Internet. Además, la integración de servicios externos como Gemini, Wger y Open Food Facts amplía las funcionalidades disponibles en la aplicación.

3.7.2 Viabilidad económica

La viabilidad económica del proyecto tiene como objetivo estimar los recursos económicos necesarios para el desarrollo de FitVive y determinar si su realización resulta asumible desde el punto de vista económico. Para ello, se consideran tanto los costes asociados a las tecnologías, herramientas e infraestructura utilizadas como el coste estimado del trabajo necesario para llevar a cabo el proyecto.

En primer lugar, se han identificado las principales tecnologías y herramientas empleadas durante el desarrollo de FitVive. Como se muestra en la Tabla 9, todas ellas se han utilizado mediante versiones gratuitas o soluciones que no han supuesto un coste económico directo para el proyecto.

Herramienta/ Recurso	Uso en el proyecto	Coste
Android Studio	Desarrollo de la aplicación	Gratuito
Kotlin Multiplatform	Desarrollo multiplataforma	Gratuito
Compose Multiplatform	Interfaz de usuario	Gratuito
SQLDelight	Base de datos local	Gratuito
Cloud Firestore	Sincronización parcial (Android)	Plan gratuito
Gemini API	Asistente conversacional	Plan gratuito
Wger API	Información de ejercicios	Gratuito
Open Food Facts	Información nutricional	Gratuito

Tabla 9. Costes asociados a las tecnologías, herramientas e infraestructura utilizada durante el desarrollo de FitVive. Fuente: Elaboración propia.

Como puede observarse, el coste directo asociado al uso de las tecnologías, herramientas e infraestructura seleccionadas ha sido de 0 €. No obstante, este valor no representa el coste económico total del proyecto, ya que también debe considerarse el trabajo dedicado a su desarrollo.

Para estimar este segundo componente se parte de las 300 horas de dedicación previstas para el desarrollo del TFG, cuya distribución entre las diferentes fases se detalla en el apartado de viabilidad temporal. Tomando como referencia una tarifa de 25 €/hora, correspondiente al límite inferior indicado para un desarrollador junior [24], el coste estimado del trabajo realizado ascendería a $300 \text{ horas} \times 25 \text{ €/hora} = 7.500 \text{ €}$.

Considerando conjuntamente ambos componentes, el coste económico estimado de FitVive sería de $0 \text{ €} + 7500 \text{ €} = 7500 \text{ €}$.

De este modo, aunque la utilización de las tecnologías y herramientas seleccionadas no ha supuesto un desembolso económico, el proyecto presenta un coste asociado al trabajo necesario para su realización. La disponibilidad de herramientas gratuitas ha permitido, por tanto, reducir los costes directos y concentrar el coste estimado del proyecto fundamentalmente en las horas dedicadas a su desarrollo.

3.7.3 Viabilidad temporal

La viabilidad temporal del proyecto tiene como objetivo determinar si el tiempo disponible para la realización del TFG permite abordar las diferentes fases previstas y alcanzar los objetivos establecidos.

Para el desarrollo de FitVive se ha considerado una dedicación total de 300 horas, distribuidas entre las fases de investigación y análisis, diseño, desarrollo e implementación, pruebas y documentación. La Tabla 10 recoge la distribución estimada de esta dedicación entre las distintas fases del proyecto.

Fase del proyecto	Horas estimadas
Investigación y análisis	40
Diseño	45
Desarrollo e implementación	140
Pruebas	35
Documentación	40
Total	300 h

Tabla 10. Distribución estimada a las horas dedicadas a cada fase del proyecto. Fuente: Elaboración propia.

Como puede observarse, la fase de desarrollo e implementación concentra la mayor parte de la dedicación prevista, con 140 horas, aproximadamente un 47 % del total. Las 160 horas restantes se distribuyen entre las actividades de investigación y análisis, diseño, pruebas y elaboración de la documentación.

Esta distribución permite planificar las diferentes actividades dentro de las 300 horas disponibles y asignar una mayor dedicación a la implementación, sin dejar de contemplar las restantes fases necesarias para el desarrollo del proyecto. Asimismo, la estimación

temporal constituye la base para calcular el coste de desarrollo considerado en el análisis de viabilidad económica.

4 Metodología

En este capítulo se presenta y justifica la metodología utilizada durante el desarrollo de FitVive, explicando cómo se ha planificado y organizado el proyecto. Se describe el proceso seguido desde la planificación inicial hasta la implementación y validación de la aplicación.

4.1 Metodología y desarrollo

Para el desarrollo de la aplicación FitVive, se ha optado por un enfoque de desarrollo incremental e iterativo. Este enfoque ha permitido desarrollar la aplicación de forma progresiva, incorporando nuevas funcionalidades y comprobando su funcionamiento antes de continuar con la siguiente fase del desarrollo.

El desarrollo incremental consiste en construir la aplicación de manera progresiva, incorporando funcionalidades nuevas sobre una versión ya existente de la aplicación. Cada incremento aporta más capacidades al sistema, permitiendo disponer en todo momento de una aplicación funcional. Por su parte, el desarrollo iterativo permite revisar y mejorar funcionalidades ya implementadas, realizando modificaciones y correcciones cuando es necesario para adaptarlas a los nuevos requisitos o mejorar su funcionamiento.

En el caso de FitVive, el desarrollo ha comenzado con la planificación y preparación del proyecto, así como el desarrollo de la estructura básica y la configuración de la aplicación. Posteriormente, se han implementado las funcionalidades principales, y una vez establecidas estas, se han incorporado las funcionalidades más específicas, como los módulos de entrenamiento, alimentación, seguimiento del progreso, el asistente conversacional basado en inteligencia artificial y el sistema de notificaciones.

Finalmente se han realizado las pruebas y mejoras necesarias sobre las funcionalidades desarrolladas, corrigiendo los errores y optimizando aquellas funcionalidades que lo requerían.

La muestra de forma esquemática las principales fases seguidas durante el desarrollo de FitVive.

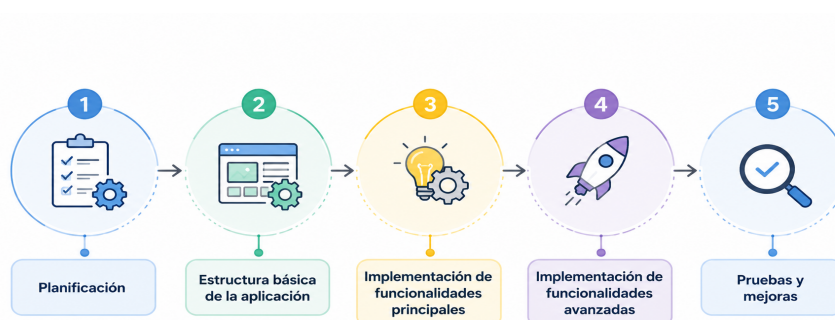


Figura 8. Fases del desarrollo de FitVive. Fuente: Elaboración propia.

La utilización de esta metodología ha aportado diversas ventajas durante el desarrollo del proyecto. Al trabajar de forma incremental, ha sido posible validar el funcionamiento de cada módulo antes de continuar con el siguiente, detectando los posibles errores durante las pruebas y reduciendo así el impacto de cambios importantes. Además, el desarrollo iterativo ha permitido mejorar funcionalidades ya desarrolladas e incorporar nuevas sin afectar a la estructura general de la aplicación, manteniendo un desarrollo ordenado y una versión funcional del proyecto en cada etapa.

4.2 Planificación

Con el objetivo de organizar el desarrollo de FitVive de forma ordenada, se ha realizado una planificación inicial en la que se han establecido las principales fases del proyecto.

Aunque se ha establecido una organización previa del trabajo, esta planificación fue adaptándose conforme avanzaba el desarrollo, incorporando nuevas funcionalidades y realizando mejoras sobre las ya implementadas. De este modo, ha sido posible mantener un desarrollo organizado y ajustado a las necesidades que han ido surgiendo durante la realización del proyecto.

El desarrollo del proyecto ha comenzado aproximadamente en el mes de abril con la definición de los objetivos, la elección de las tecnologías y la configuración del entorno de desarrollo. En esta etapa se ha establecido la estructura base de la aplicación y se ha definido la organización general del proyecto.

Entre los meses de mayo y junio se ha desarrollado la mayor parte de las funcionalidades de FitVive, incorporando progresivamente los diferentes módulos de la aplicación, como el sistema de autenticación, el perfil de usuario, los entrenamientos, la alimentación, el seguimiento del progreso, el asistente conversacional basado en inteligencia artificial y el sistema de notificaciones. Durante el desarrollo se han realizado pruebas y se ha validado el correcto funcionamiento de la aplicación, corrigiendo los errores detectados. Una vez finalizado el desarrollo de la aplicación, se ha iniciado la redacción de la memoria del Trabajo Fin de Grado, documentando el proceso seguido y las decisiones adoptadas durante el proyecto.

La tabla 11 muestra la planificación temporal seguida durante el desarrollo del proyecto.

Mes	Actividades
Abril	Planificación del proyecto, definición de objetivos, selección de tecnologías y configuración inicial.
Mayo	Desarrollo de la estructura base e implementación de las funcionalidades principales.
Junio	Desarrollo de las funcionalidades restantes, pruebas y validación de la aplicación.
Julio	Redacción de la memoria del TFG y revisión del contenido.
Agosto	Corrección de errores, incorporación de mejoras finales, revisión de memoria y preparación de la entrega definitiva

Tabla 11. Planificación del proyecto. Fuente: Elaboración propia.

Para representar de forma más clara la planificación del proyecto, se ha elaborado un cronograma dividido en diferentes iteraciones. Cada iteración incluye tareas de análisis, diseño, desarrollo y pruebas sobre las funcionalidades correspondientes. Además, se incluyen las pruebas generales, las mejoras realizadas y la redacción y revisión de la memoria. La tabla 12 muestra la distribución de estas actividades durante el desarrollo del proyecto.

Actividad	Abril	Mayo	Junio	Julio	Agosto
Planificación y configuración inicial					
Iteración 1. Estructura base, autenticación y perfil.					
Iteración 2. Entrenamientos y alimentación.					
Iteración 3. Progreso, asistente IA y notificaciones					
Pruebas generales y mejoras.					
Redacción y revisión de la memoria					

Tabla 12. Cronograma de planificación y desarrollo de FitVive. Fuente: Elaboración propia.

5 Diseño e implementación

En este capítulo se presenta el diseño y desarrollo de FitVive. Para ello, se describe la arquitectura general de la aplicación, el modelo de datos, el diseño de la interfaz de usuario y la integración con servicios externos. Asimismo, se detalla cómo se usan lenguajes y frameworks con Kotlin, y se detallan las principales funcionalidades implementadas, las diferentes pantallas de la aplicación y su flujo de navegación. Finalmente, se justifican las principales decisiones técnicas y de diseño adoptadas durante el desarrollo del proyecto.

5.1 Diseño del sistema

En este apartado se describe el diseño general del sistema, incluyendo la arquitectura utilizada, la organización de los principales componentes y la forma en la que interactúan la interfaz de usuario, la persistencia local y los servicios externos. También se explica la separación entre el código compartido y las implementaciones específicas de Android e iOS.

5.1.1 Arquitectura de la aplicación

FitVive ha sido desarrollada como una aplicación multiplataforma utilizando Kotlin Multiplatform, lo que permite compartir gran parte de la lógica entre Android e iOS. La arquitectura del proyecto se ha desarrollado de forma modular, separando las distintas responsabilidades de la aplicación para facilitar el mantenimiento, escalabilidad y reutilización del código. La aplicación utiliza una organización basada en Compose, con una separación entre la capa de presentación, el acceso a los datos y los servicios externos.

La capa de presentación está formada por las diferentes funciones componibles (*composables*) que representan las pantallas de la aplicación. El componente principal “App” actúa como eje central de coordinación, ya que mantiene la información necesaria para el funcionamiento de la aplicación, gestiona la navegación entre pantallas y distribuye los datos y las acciones necesarias a los diferentes componentes de la interfaz.

El acceso a la base de datos local se realiza mediante repositorios. `UsuarioRepository` almacena las operaciones relacionadas con la base de datos `SQLDelight`, incluyendo la gestión de usuarios, entrenamientos, registros de peso, notificaciones y comidas completadas. Por otro lado, `ApiRepository` se encarga de obtener la información necesaria para determinadas funcionalidades de la aplicación.

Para ello, primero consulta los datos almacenados en la base de datos local, y si estos no son válidos o no están disponibles, realiza una petición a las APIs externas. La información obtenida se almacena en caché para poder reutilizarla en futuras consultas, reduciendo de esta manera las peticiones a las APIs.

La comunicación con los servicios externos se implementa mediante Ktor. Para ello, la aplicación utiliza clases de servicio que son las encargadas de comunicarse con las APIs externas, siendo necesaria la conexión a Internet para el correcto funcionamiento. Estas

permiten obtener información sobre ejercicios y alimentación, así como enviar consultas al asistente basado en Gemini y recibir sus respuestas.

Kotlin Multiplatform permite compartir gran parte del código entre Android e iOS. Sin embargo, cuando existe alguna funcionalidad que depende del sistema operativo, se crea una implementación distinta para cada plataforma, como el acceso a la base de datos, la gestión de las notificaciones, la carga de imágenes y la sincronización con Firestore.

La arquitectura de FitVive no sigue de manera estricta patrones como MVVM o UDF (Unidirectional Data Flow). No se utiliza una estructura MVVM, ya que la aplicación no dispone de un ViewModel específico para cada pantalla. El componente principal App gestiona gran parte de la información y la navegación de la aplicación, mientras que algunas pantallas gestionan sus propios datos.

Sin embargo, la aplicación utiliza parcialmente algunos principios de UDF. El componente App envía la información necesaria a las pantallas y recibe mediante callbacks los cambios realizados por el usuario para actualizar los datos y la interfaz. Por otro lado, los repositorios gestionan el acceso a los datos y los servicios se encargan de la comunicación con las APIs externas.

La Figura 9 muestra la organización general de la aplicación y la relación entre la interfaz, los repositorios, los servicios externos y la base de datos local.

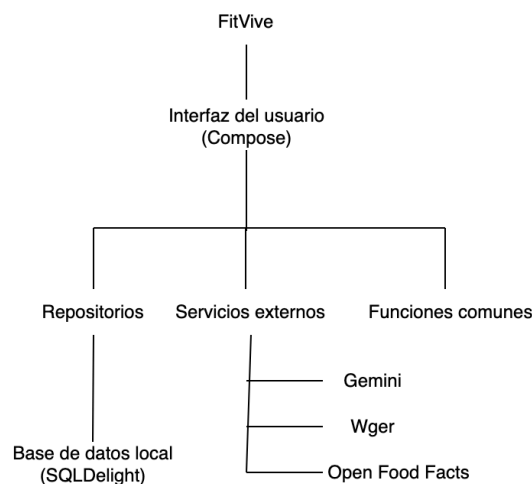


Figura 9. Arquitectura general de FitVive. Fuente: Elaboración propia.

5.1.2 Organización de los componentes

La organización de los componentes de FitVive sigue la estructura propuesta por Kotlin Multiplatform, diferenciando el código compartido del código específico de cada plataforma. Esta organización permite reutilizar la mayor parte de la aplicación entre Android e iOS y mantener separadas aquellas funcionalidades que requieren una implementación distinta. A continuación, se describen los principales componentes que forman parte del proyecto y la función que desempeña cada uno.

5.1.2.1 Código compartido

El código compartido de FitVive se encuentra en `commonMain`, que es el núcleo de la aplicación desarrollada con Kotlin Multiplatform. En este directorio se implementa la mayor parte de la funcionalidad, permitiendo reutilizar el mismo código tanto en Android como en iOS.

Dentro de este código compartido se incluyen pantallas desarrolladas con Compose Multiplatform, la gestión de la navegación, la lógica principal de la aplicación, el acceso a la base de datos, la comunicación con los servicios externos.

El componente principal de este directorio es “App”, que es el encargado de coordinar el funcionamiento general de la aplicación. Desde este componente se controla la navegación entre pantallas y la comunicación con los diferentes componentes que forman parte de la aplicación.

Gracias a esta organización, la mayor parte del desarrollo se realiza una única vez y puede ejecutarse en ambas plataformas, evitando tener que desarrollar las mismas funcionalidades por separado y facilitando su mantenimiento.

La Figura 10 muestra de forma esquemática cómo se ve la organización del código compartido de `commonMain`.

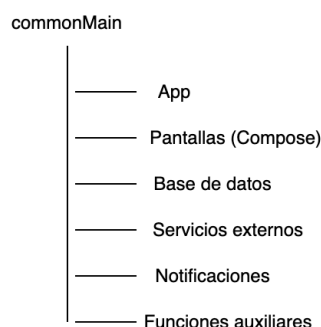


Figura 10. Organización código compartido (commonMain). Fuente: Elaboración propia.

5.1.2.2 Persistencia de datos

La persistencia de los datos de FitVive se gestiona mediante SQLDelight, una librería que permite almacenar y consultar información en una base de datos SQLite desde el código compartido de la aplicación. De este modo, la aplicación puede almacenar información de forma local tanto en Android como en iOS, aunque cada plataforma utiliza su propia implementación del controlador de la base de datos.

El acceso a los datos se concentra principalmente en `UsuarioRepository`, que actúa como intermediario entre el resto de la aplicación y la base de datos. Este componente reúne las operaciones necesarias para guardar, consultar y actualizar información relacionada con el usuario, los entrenamientos realizados, los registros de peso, la configuración de las notificaciones y las comidas completadas.

Por otra parte, `ApiRepository` se encarga de gestionar la información obtenida de los servicios externos junto con los datos almacenados en la base de datos local. Antes de realizar una consulta a los servicios externos, comprueba si existen datos disponibles en la caché local. Cuando obtiene nueva información, esta puede almacenarse en la base de datos para reutilizarla posteriormente, reduciendo así el número de peticiones a dichos servicios.

5.1.2.3 Comunicación con servicios externos

La comunicación con los servicios externos se concentra en el paquete “network”, que utiliza Ktor para realizar las peticiones HTTP a las diferentes API empleadas por la aplicación.

A través de estos servicios, FitVive obtiene información sobre ejercicios y alimentos, además de gestionar las consultas realizadas al asistente basado en inteligencia artificial. Para ello, la aplicación se comunica con las API de Wger, Open Food Facts y Gemini, procesando posteriormente la información recibida para después utilizarla en las diferentes funcionalidades.

Los servicios relacionados con los ejercicios y la alimentación trabajan juntamente con `ApiRepository`, que se encarga de consultar la caché local antes de realizar una petición externa y de almacenar los nuevos datos cuando es necesario. Por otra parte, el servicio de asistente se utiliza directamente desde la pantalla correspondiente, ya que las respuestas se generan en cada consulta y no se almacenan en la base de datos.

5.1.2.4 Componentes específicos de plataforma

Aunque la mayor parte de la aplicación se desarrolla en el código compartido, algunas funcionalidades requieren una implementación específica para cada sistema operativo. Estas se encuentran en los módulos `androidMain` e `iOSMain`, donde se incluyen las implementaciones que requieren utilizar funciones propias de cada plataforma.

Entre estas implementaciones se encuentran el acceso a la base de datos, la gestión de las notificaciones y otras funcionalidades que dependen del sistema operativo. Gracias a esta separación, el código específico de cada plataforma se mantiene aislado del código compartido.

5.1.3 Flujo general de funcionamiento

El funcionamiento de FitVive comienza con el inicio de la aplicación, donde se inician los componentes necesarios para la gestión de datos, la navegación y la comunicación con los servicios externos. Además, se recupera la información almacenada localmente mediante `SQLDelight`, lo que permite recuperar el estado del usuario y decidir qué pantalla debe mostrarse en primer lugar.

Una vez completada la carga inicial, la aplicación comprueba si el usuario ha realizado previamente la configuración inicial. En caso contrario, se muestra la pantalla de bienvenida para iniciar el registro y completar los datos necesarios del perfil. Si el usuario ya dispone de un perfil configurado, se accede directamente a la pantalla principal de la aplicación.

Desde la pantalla principal, el usuario puede acceder a las distintas funcionalidades disponibles, como la gestión de entrenamientos, la planificación de la alimentación, el seguimiento del progreso, la configuración del perfil, y el asistente basado en inteligencia artificial. Cada una de estas secciones obtiene la información necesaria desde la base de datos local o, cuando es necesario, mediante los servicios externos integrados en la aplicación. Las modificaciones realizadas se almacenan localmente y, cuando es necesario, también se sincronizan.

Todo este flujo se encuentra coordinado desde `App.kt`, que gestiona la navegación entre pantallas, mantiene el estado compartido de la aplicación y coordina la comunicación entre los distintos componentes.

La Figura 11 resume el flujo general de funcionamiento de FitVive desde el inicio de la aplicación hasta la interacción del usuario con las distintas funcionalidades. En ella se muestran las etapas más importantes del proceso, incluyendo la carga inicial de datos, la selección de la pantalla de inicio o la navegación entre las diferentes secciones de la aplicación.

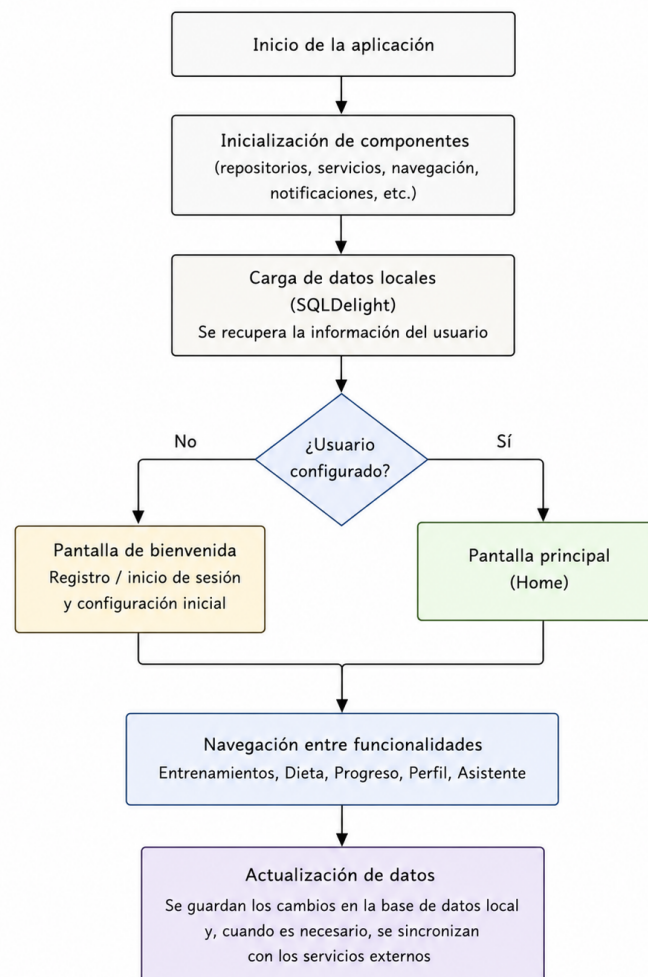


Figura 11. Flujo general de funcionamiento de FitVive. Fuente: Creada con IA Gen.

5.1.4 Integración multiplataforma

FitVive se ha desarrollado utilizando Kotlin Multiplatform, lo que permite compartir la mayor parte del código entre Android e iOS. La lógica de la aplicación, la interfaz de usuario creada con Compose Multiplatform, los repositorios, la persistencia mediante SQLDelight y la comunicación con los servicios externos se encuentran en el módulo “commonMain”, por lo que únicamente se desarrollan una vez y pueden utilizarse en ambas plataformas.

Las funcionalidades que dependen del sistema operativo se implementan en los módulos “androidMain” e “iosMain”. Entre ellas se encuentran el inicio de la aplicación, el acceso a la base de datos mediante los controladores específicos de SQLite, la gestión de notificaciones y otras funcionalidades propias de cada plataforma.

La reutilización del código se consigue manteniendo en `commonMain` todos los componentes que son independientes de la plataforma. Cuando una funcionalidad depende del sistema operativo, se proporciona una implementación específica para Android e iOS, manteniendo así una única lógica de funcionamiento para ambas plataformas.

Aunque la mayor parte del comportamiento en FitVive es idéntico en ambas plataformas, existen algunas diferencias en la implementación. La más relevante es que la sincronización con Firestore está disponible únicamente en Android, mientras que iOS utiliza una implementación sin sincronización. Del mismo modo, cada plataforma utiliza sus propias API para gestionar las notificaciones y el acceso a la base de datos.

La Figura 12 muestra de forma esquemática cómo se organiza el código de FitVive entre el módulo compartido y los módulos específicos de Android e iOS, indicando los principales componentes que se implementan en cada uno de ellos.

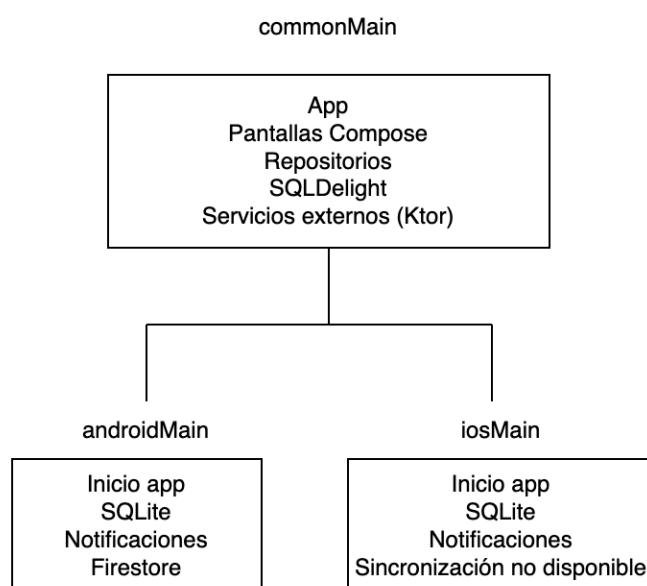


Figura 12. Organización del código compartido y específico de cada plataforma en FitVive. Fuente: Elaboración propia.

5.2 Diseño de la interfaz

En este apartado se describe el diseño de la interfaz de usuario de FitVive, explicando los criterios seguidos durante su desarrollo, la organización de la navegación y las principales pantallas que componen la aplicación. El objetivo es ofrecer una experiencia de uso intuitiva, sencilla y coherente, que esté adaptada a las necesidades del usuario.

5.2.1 Criterios de diseño

El diseño de la interfaz de FitVive se ha desarrollado con el objetivo de ofrecer una experiencia de uso intuitiva, coherente y sencilla. Al tratarse de una aplicación orientada al seguimiento de entrenamientos, la alimentación y progreso físico, se ha buscado que el acceso a estas funcionalidades sea claro, permitiendo al usuario localizar rápidamente la información que necesita.

Para mantener un diseño visual uniforme, se ha optado por que todas las pantallas compartan un mismo estilo, basada en un fondo oscuro, superficies en tonos grises, y el color naranja para destacar botones, indicadores y acciones principales. Además, las distintas pantallas siguen una organización similar, situando los títulos en la parte superior de la pantalla y distribuyendo el contenido principal en bloques claramente diferenciados. Esta disposición facilita la navegación, proporcionando una experiencia de uso más uniforme.

Otro de los criterios seguidos durante el diseño ha sido la reutilización de los elementos comunes en las distintas pantallas. Componentes como la barra de navegación inferior, las tarjetas, los botones o los formularios mantienen un mismo estilo visual en toda la aplicación, proporcionando una mayor coherencia y facilitando el uso de las diferentes funcionalidades.

Finalmente, el diseño ha adaptado a las características de Compose Multiplatform, permitiendo compartir la mayor parte de la interfaz entre Android e iOS sin modificar la experiencia de uso. De este modo, ambas plataformas presentan la misma organización de pantallas, navegación y estilo visual, garantizando un comportamiento uniforme para el usuario.

5.2.2 Organización de la navegación

La navegación de FitVive se ha diseñado para que el usuario pueda acceder de forma sencilla a las principales funcionalidades de la aplicación. Tras la pantalla de bienvenida, el usuario puede iniciar sesión o registrarse y, en caso de acceder por primera vez, completar un proceso de configuración inicial en el que se introducen datos como el objetivo físico, los días de entrenamiento o el grupo muscular prioritario. Una vez finalizado este proceso, se accede a la pantalla principal de la aplicación.

Las funcionalidades principales se organizan mediante una barra de navegación situada en la parte inferior de la pantalla, que permanece disponible en las secciones principales de la aplicación. A través de ella es posible acceder directamente a las pantallas de Inicio, Entrenamiento, Dieta y Perfil. Desde la pantalla de inicio también se puede acceder a funcionalidades complementarias, como el seguimiento del progreso y el asistente basado en inteligencia artificial.

Además de esta navegación principal, algunas funcionalidades disponen de pantallas secundarias a las que se accede desde la pantalla principal correspondiente. Por ejemplo, desde la sección de entrenamientos es posible consultar el detalle de cada sesión, mientras que desde la sección de alimentación se accede a la planificación diaria de las comidas. Del mismo modo, la pantalla de progreso permite registrar nuevas mediciones de peso, mientras que la pantalla de perfil ofrece acceso a la configuración de las notificaciones.

Con el fin de facilitar la comprensión de la organización de la navegación de la aplicación, la Figura 13 muestra las principales pantallas que componen FitVive y las relaciones de navegación existente entre ellas. En ella se representa el flujo de acceso desde la pantalla de bienvenida hasta la pantalla de inicio, así como las distintas secciones accesibles mediante la navegación principal y sus correspondientes pantallas secundarias

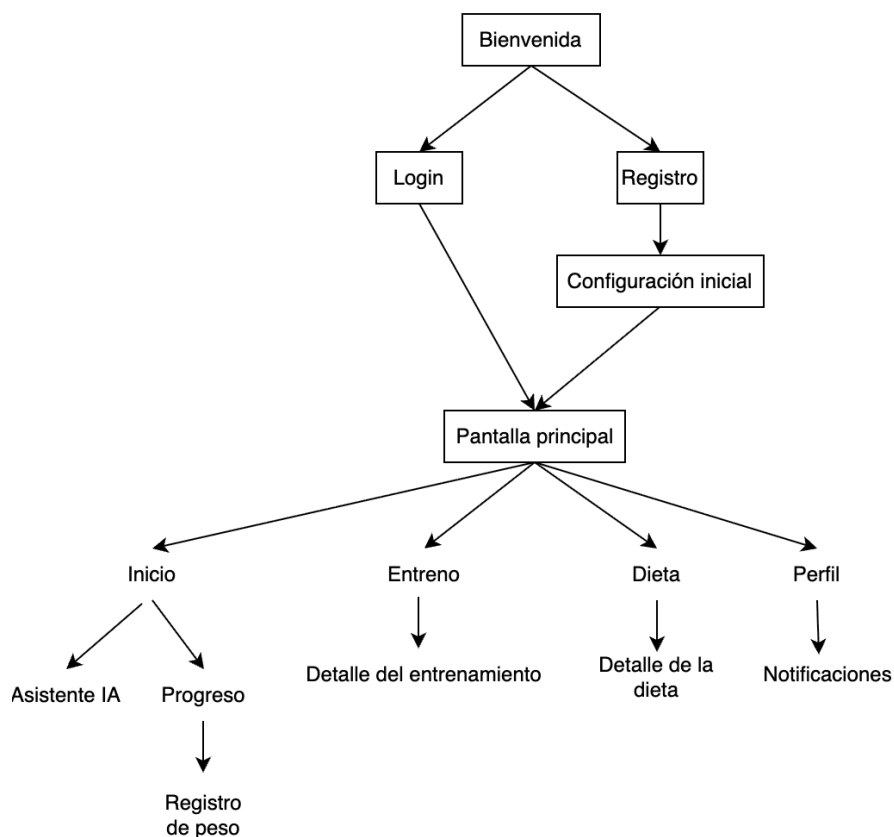


Figura 13. Mapa de navegación de las principales pantallas de FitVive. Fuente: Elaboración propia.

5.2.3 Diseño de la interfaz del usuario

Una vez descritos los criterios de diseño y la organización de la navegación de la aplicación, en este apartado se describen las principales pantallas que componen la interfaz de FitVive. En cada una de ellas se explica su propósito, su organización y sus funcionalidades más relevantes.

5.2.3.1 Pantalla de bienvenida

La pantalla de bienvenida es el punto de acceso a FitVive y es la primera pantalla que se muestra al iniciar la aplicación. Su objetivo es ofrecer una interfaz intuitiva y sencilla para el proceso de autenticación, permitiendo al usuario registrarse o iniciar sesión con una cuenta ya existente.

Se ha optado por una interfaz minimalista, evitando elementos innecesarios que puedan confundir al usuario. La aplicación mantiene la identidad visual de FitVive mediante el uso del logotipo, con un fondo oscuro y detalles en color naranja. La Figura 14 muestra el diseño final del logotipo.



Figura 14. Logotipo de FitVive. Fuente: Elaboración propia.

Desde esta pantalla el usuario puede acceder tanto al formulario de inicio de sesión como al proceso de registro, que da paso a la configuración inicial de la aplicación.

Antes de la implementación de la interfaz se ha realizado un diseño previo mediante Figma. Este diseño ha servido como referencia inicial durante el desarrollo, aunque posteriormente se han realizado modificaciones en el estilo visual y en la organización de algunos elementos. La Figura 15 muestra el diseño previo de la pantalla de bienvenida.

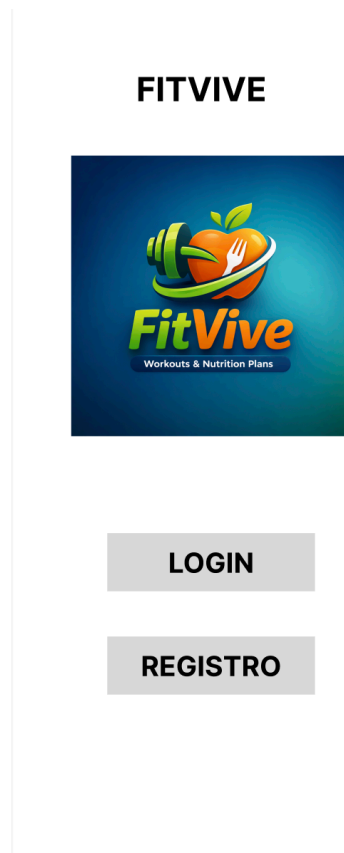


Figura 15. Diseño previo de la pantalla de bienvenida de FitVive. Fuente: Elaboración propia.

5.2.3.2 Configuración inicial

La pantalla de configuración inicial permite recopilar la información necesaria para el funcionamiento de FitVive. Este proceso se muestra únicamente la primera vez que el usuario accede a la aplicación tras completar el registro.

Con el objetivo de facilitar la configuración del perfil, se organiza en cuatro pantallas consecutivas. En la primera se solicitan los datos personales y físicos del usuario, como el sexo, el peso, la altura y la fecha de nacimiento. A continuación, el usuario selecciona su objetivo físico, el número de días de entrenamiento semanales y el grupo muscular al que desea dar prioridad. Esta organización evita presentar un formulario extenso en una única pantalla y hace que el proceso sea más sencillo e intuitivo.

Durante el proceso se realizan distintas validaciones para comprobar que los datos introducidos sean correctos antes de permitir avanzar al siguiente paso. Una vez completada la configuración, la información queda almacenada en el perfil del usuario y se utilizará posteriormente para personalizar los planes de entrenamiento y alimentación.

Antes de implementar el proceso de configuración inicial, se ha realizado un diseño previo de las diferentes pantallas que lo componen. En este diseño define la distribución de los campos necesarios para recopilar la información del usuario. La Figura 16 muestra el diseño previo realizado en Figma.

El diagrama muestra el flujo de la configuración inicial de FitVive, dividido en cinco secciones principales:

- Login:** Campos para "Correo electrónico" y "Contraseña", con un botón "ENTRAR".
- Crear cuenta:** Campos para "Nombre", "Apellidos", "Correo electrónico" y "Contraseña", con un botón "Registrarse".
- PASO 1:** Campos para "SEXO" (selección), "PESO" (con unidad "KG"), "ALTURA" (con unidad "CM") y "FECHA NAC" (con campos para "MES" y "AÑO"). Incluye un botón "SIGUIENTE PASO".
- PASO 2:** Sección "Objetivos" con el texto "Elige una opción" y tres botones: "VOLUMEN", "DÉFICIT" y "MANTENIMIENTO". Incluye un botón "SIGUIENTE PASO".
- PASO 3:** Sección "Días de entrenamiento a la semana" con un menú desplegable "Seleccione" que muestra opciones 3, 4, 5 y 6. Incluye un botón "Comencemos".

Figura 16. Diseño previo del proceso de configuración inicial de FitVive. Fuente: Elaboración propia.

5.2.3.3 Pantalla principal

La pantalla principal es el punto de acceso a las funcionalidades de FitVive y es la primera que se muestra una vez que el usuario ha iniciado sesión o completado la configuración inicial. Se ha optado por un acceso rápido e intuitivo a las distintas secciones de la aplicación.

En la parte superior se muestra un saludo personalizado con el nombre del usuario, acompañado de un mensaje motivacional "Listo para entrenar hoy". Además, aparece el logotipo de la aplicación y un bloque gráfico con una ilustración relacionada con el entrenamiento acompañado de un lema, con la finalidad de ayudar al usuario a mantener una rutina de entrenamiento.

Debajo se incluye un indicador con el número de días de entrenamiento semanales ya configurado previamente por el usuario en el proceso de configuración inicial.

El contenido principal está formado por cuatro bloques que permiten acceder a las diferentes secciones de Entrenamiento, Dieta personalizada, Mi progreso y Asistente IA. Cada una de ellas incorpora un icono identificativo y una breve descripción.

En la parte inferior de la pantalla se encuentra la barra de navegación, desde la que el usuario puede acceder a las secciones Inicio, Entreno, Dieta y Perfil.

Antes de implementar la pantalla principal, se ha realizado un diseño previo que ha servido de referencia inicial, aunque posteriormente se han realizado algunos cambios en el diseño. La Figura 17 muestra el diseño previo de la pantalla principal.

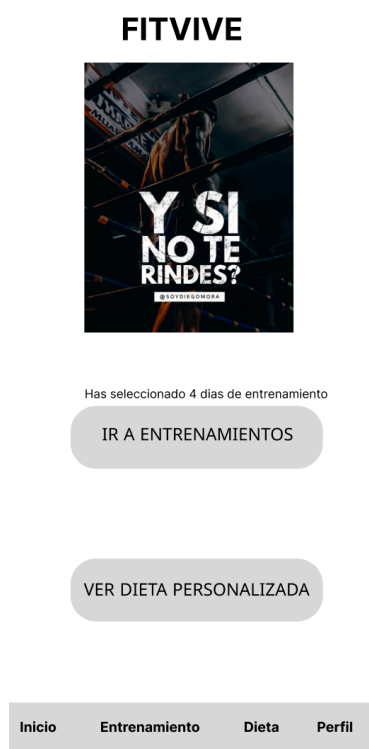


Figura 17. Diseño previo de la pantalla principal de FitVive realizado en Figma. Fuente: Elaboración propia.

5.2.3.4 Entrenamientos

La pantalla de entrenamientos permite al usuario consultar el plan de entrenamiento personalizado generado por la aplicación y registrar la realización de cada sesión. Se puede acceder a ella desde la pantalla principal mediante la sección Entrenamientos, o desde la barra de navegación inferior seleccionando la opción Entreno.

La interfaz muestra una lista con los distintos días de entrenamiento disponibles. Al seleccionar uno de ellos, se accede al detalle de la rutina correspondiente, donde se presentan los distintos ejercicios con la información necesaria para su realización.

Además, el usuario puede registrar los datos de cada ejercicio realizado y marcar el entrenamiento como completado. La planificación se adapta a las características y objetivos del usuario definidos en la configuración inicial.

Para la sección de Entrenamientos, se ha planteado una interfaz sencilla a partir de la cual se fue desarrollando y modificando dicha pantalla. La Figura 18 muestra el diseño previo realizado en Figma.

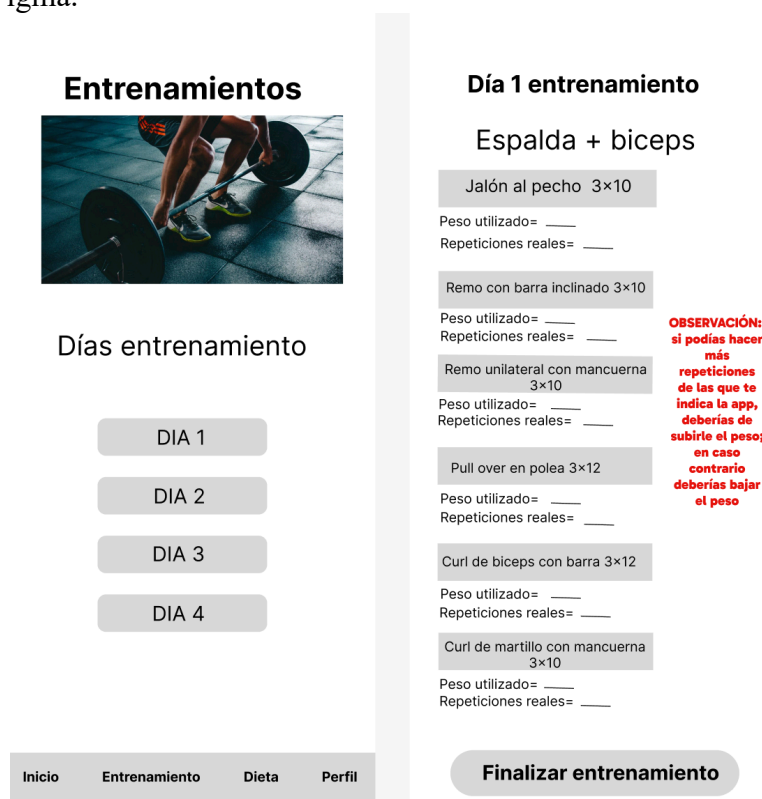


Figura 18. Diseño previo de la pantalla de entrenamientos de FitVive. Fuente: Elaboración propia.

5.2.3.5 Dieta

La funcionalidad de dieta permite al usuario consultar un plan de alimentación personalizado generado por la aplicación. Se puede acceder a esta sección desde el bloque Dieta personalizada de la pantalla principal o mediante la opción Dieta en la barra de navegación inferior.

La interfaz muestra una lista con los diferentes días de la semana, permitiendo seleccionar el día que se desee. Al acceder a un día en concreto, se presenta un detalle de las comidas planificadas, organizadas en desayuno, comida, merienda y cena, junto con la información necesaria para su consulta.

El plan alimentario se adapta al objetivo físico seleccionado durante la configuración inicial, ofreciendo una planificación acorde con las necesidades establecidas por el usuario.

Además, el usuario puede marcar la comida como completada para realizar un seguimiento de su cumplimiento.

La figura 19 muestra un diseño de la interfaz propuesta previa al desarrollo de la aplicación.

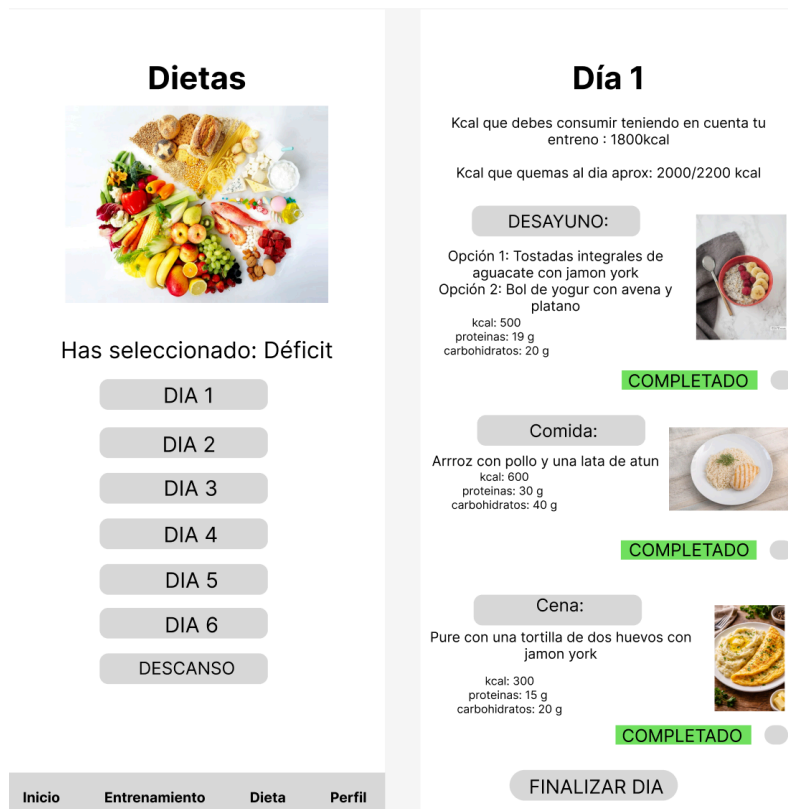


Figura 19. Diseño previo de la pantalla de Dieta de FitVive. Fuente: Elaboración propia.

5.2.3.6 Progreso

La pantalla de progreso permite al usuario consultar la evolución de su peso y realizar un seguimiento de los registros almacenados en la aplicación. Se puede acceder a esta sección desde el bloque Progreso en la pantalla principal.

La interfaz muestra un resumen con información relevante del usuario, como el peso actual, el número de días consecutivos de entrenamiento y el grupo muscular prioritario configurado. Además, incorpora una gráfica que representa la evolución del peso a partir de los registros realizados, permitiendo visualizar los cambios experimentados a lo largo del tiempo.

Debajo de la gráfica se presenta el historial de registros de peso, desde el que el usuario puede editar o eliminar cualquiera de los registros almacenados.

Esta sección permite también añadir un nuevo registro, cuya funcionalidad permite introducir su peso actual, valorar la evolución de la semana y añadir observaciones.

La pantalla de progreso se ha incorporado durante el desarrollo de la aplicación, por lo que no se ha realizado un diseño previo en Figma. Su interfaz se ha creado directamente siguiendo el mismo estilo que el resto de FitVive.

5.2.3.7 Asistente IA

El Asistente IA permite al usuario realizar consultas relacionadas con el entrenamiento, la alimentación y los hábitos saludables. Se puede acceder a esta funcionalidad desde el bloque Asistente IA de la pantalla principal.

La interfaz está organizada como un chat de conversación. Está formada por una cabecera con un mensaje de bienvenida, un área donde se muestran los mensajes de la conversación y un campo de texto con un botón de envío para realizar consultas.

Las respuestas se generan teniendo en cuenta la información básica del perfil del usuario, como el objetivo físico, el peso actual y el número de días de entrenamiento semanales, con el fin de ofrecer recomendaciones más adaptadas a sus características.

El asistente basado en inteligencia artificial se ha incorporado más adelante en el proyecto, por lo que tampoco se ha realizado un diseño previo en Figma. La pantalla se ha diseñado de forma sencilla, siguiendo una estructura similar a la de un chat.

5.2.3.8 Perfil

La pantalla de perfil permite al usuario consultar su información personal y deportiva, modificar determinados datos de configuración y gestionar los recordatorios de la aplicación. Se puede acceder a esta sección desde la opción Perfil en la barra de navegación inferior.

La interfaz muestra un resumen con la información del usuario, incluyendo sus datos personales y de entrenamiento. Además, incorpora opciones para editar el perfil, acceder a la configuración de notificaciones, volver a la pantalla principal y cerrar sesión.

La edición del perfil se realiza a través de un formulario desde el que es posible actualizar los datos del usuario. La aplicación dispone, además, de una pantalla específica para configurar los recordatorios relacionados con el entrenamiento, la dieta y el registro periódico del peso, permitiendo activar o desactivar las notificaciones y configurar su horario de envío.

La Figura 20 muestra el diseño previo de la pantalla Perfil de la aplicación, siendo posteriormente modificada.

Figura 20. Diseño previo de la pantalla Perfil de FitVive. Fuente: Elaboración propia.

5.3 Modelo y persistencia de datos

En este apartado se describe el modelo de datos de FitVive y el sistema utilizado para almacenar la información de la aplicación. En primer lugar, se presenta el modelo de datos empleado por la aplicación y, posteriormente, se describen el mecanismo de persistencia utilizado para conservar la información y el proceso de sincronización implementado en la versión de Android.

FitVive almacena la información de la aplicación en diferentes entidades que representan los principales datos del sistema, como los usuarios, los entrenamientos, los registros de peso, las comidas completadas y la configuración de las notificaciones. Estos datos se guardan de forma local mediante SQLDelight sobre una base de datos SQLite, lo que permite conservar la información entre distintas ejecuciones de la aplicación.

La Figura 21 muestra el modelo entidad-relación utilizado en la aplicación. En él puede observarse que un usuario puede estar asociado a varios entrenamientos, registros de peso y comidas completadas, mientras que dispone de una única configuración de notificaciones y una única sesión activa.

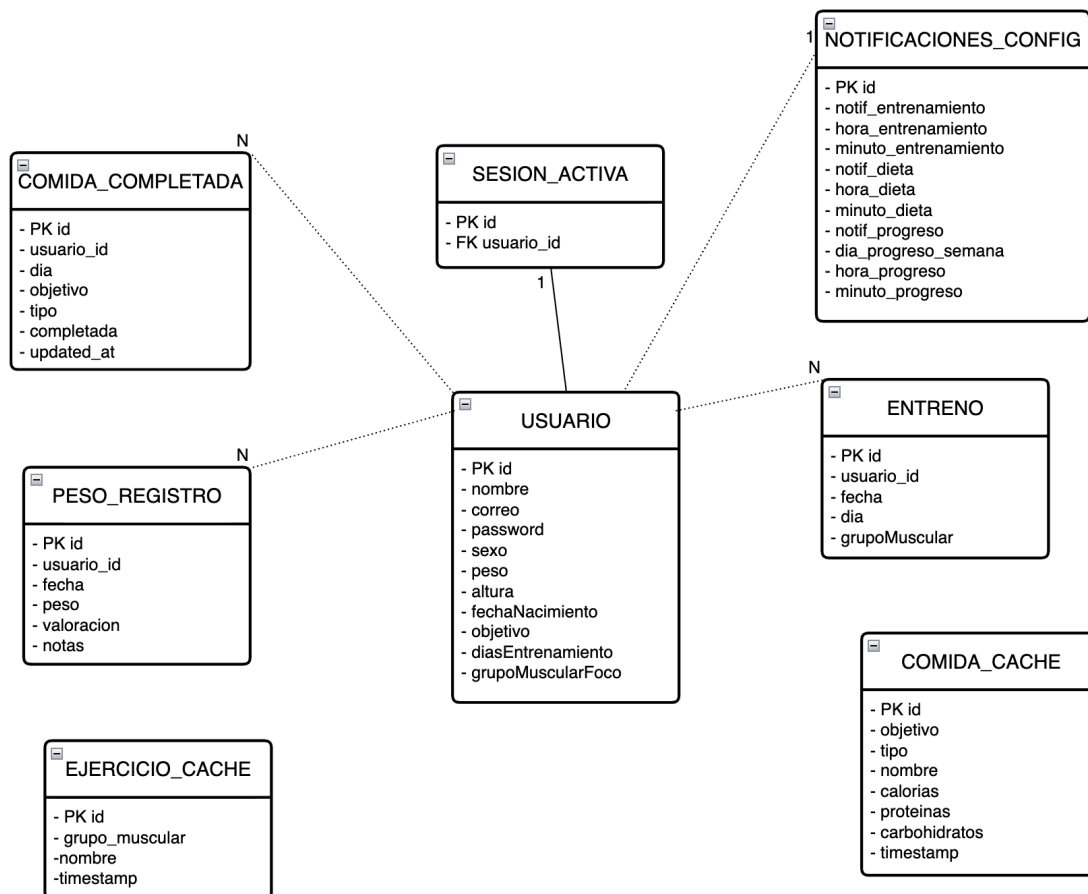


Figura 21. Diagrama Entidad-Relación. Fuente: Elaboración propia.

5.3.1 Modelo de datos

El modelo de datos de FitVive se ha diseñado tomando como elemento central la entidad **Usuario**. En ella se almacena la información necesaria para identificar a cada usuario, junto con sus datos personales y las preferencias utilizadas para personalizar los planes de entrenamiento y alimentación.

A partir de esta entidad, se organizan el resto de los datos generados durante el uso de la aplicación. La entidad **Entreno** almacena el historial de entrenamientos realizados por cada usuario, mientras que **PesoRegistro** registra los historiales de peso para facilitar el seguimiento de la evolución. Por su parte, **ComidaCompletada** almacena el estado de las comidas marcadas como realizadas dentro de la planificación diaria, permitiendo mantener dicho progreso entre distintas sesiones.

La entidad **NotificacionesConfig** almacena la configuración de las notificaciones de la aplicación, incluyendo las preferencias relacionadas con los recordatorios de entrenamiento, alimentación y seguimiento del progreso. Asimismo, **SesionActiva** identifica al usuario que mantiene iniciada la sesión, permitiendo recuperar su estado al volver a abrir la aplicación.

Además de las entidades asociadas al usuario, el modelo incorpora las tablas `EjercicioCache` y `ComidaCache`, que almacenan temporalmente la información obtenida de las APIs. Estas tablas permiten reutilizar los datos descargados anteriormente y reducir así el número de peticiones realizadas a dichos servicios. Como se trata de información temporal, no hay una relación directa con el resto de las entidades del modelo.

5.3.2 Persistencia local

La persistencia local de FitVive se ha implementado mediante `SQLDelight`, utilizando una base de datos `SQLite` para almacenar la información de la aplicación. De este modo, tanto Android como iOS comparten el mismo esquema de base de datos y la misma lógica de acceso a la información, cambiando únicamente la forma de inicializar la base de datos en cada plataforma.

La estructura de la base de datos se define mediante archivos `.sq`, donde se especifican las tablas y las consultas. A partir de estas definiciones, `SQLDelight` genera automáticamente el código necesario para acceder a la base de datos, evitando implementar manualmente las operaciones de acceso a la información y simplificando el desarrollo de la aplicación.

El acceso a la base de datos se organiza en `UsuarioRepository`, la encargada de gestionar las operaciones de almacenamiento, consulta, actualización y eliminación de la información. También administra los entrenamientos, los registros de peso, las comidas completadas, la configuración de notificaciones y las tablas de caché utilizadas para almacenar temporalmente la información obtenida de las APIs. Por ello, las distintas pantallas de la aplicación no acceden a la base de datos directamente, sino que realizan todas las operaciones a través de este repositorio. Además, dichas operaciones se ejecutan mediante corrutinas para evitar bloquear el hilo principal de la aplicación.

Para garantizar la compatibilidad entre las distintas versiones de la aplicación, se ha incorporado un sistema de migraciones de la base de datos. Estas permiten actualizar la base de datos cuando se incorporan nuevas funcionalidades, como nuevas tablas o modificaciones en las ya existentes, conservando la información almacenada por el usuario durante las actualizaciones.

5.3.3 Sincronización

Con el objetivo de disponer de una copia remota de la información más relevante del usuario, FitVive incorpora un sistema de sincronización con `Firebase Firestore`. Esta funcionalidad solo se encuentra disponible en la versión para Android, mientras que la versión para iOS la aplicación utiliza únicamente el almacenamiento local mediante `SQLDelight`, por lo que no dispone de sincronización ni de una copia remota de la información.

La sincronización se realiza después de que la información haya sido almacenada localmente en la base de datos `SQLite`. Cuando el usuario realiza determinadas acciones, como completar o modificar su perfil, registrar un entrenamiento, actualizar la configuración de notificaciones o añadir un registro de peso, los datos se guardan primero

en la base de datos local y, posteriormente se envían a Firestore para mantener una copia remota.

Los datos sincronizados incluyen la información del perfil del usuario, los entrenamientos realizados, los registros de peso y la configuración de notificaciones. Además, al finalizar un entrenamiento también se almacenan en Firestore los ejercicios realizados y las series completadas, información que no se conserva en la base de datos local y que posteriormente se utiliza para calcular las estadísticas del progreso.

La sincronización no es bidireccional. Aunque los datos se envían de la base de datos local hacia Firestore, no se realiza una descarga automática de los datos desde Firestore hacia la base de datos local.

5.4 Comunicación con servicios externos

FitVive se comunica con distintos servicios externos para ampliar la información disponible en la aplicación. Estas comunicaciones se realizan mediante Ktor Client, una biblioteca que permite compartir la lógica de las peticiones entre Android e iOS. La aplicación utiliza servicios externos para obtener información sobre ejercicios y sobre información alimentaria. Además, ha implementado un asistente conversacional relacionado con el entrenamiento y la alimentación.

5.4.1 Servicio de ejercicios

FitVive utiliza la API de wger para obtener información sobre los ejercicios asociados a cada entrenamiento. Esta API proporciona datos como el nombre del ejercicio, el grupo muscular trabajado, el equipamiento necesario y cuando está disponible una imagen representativa.

La consulta se realiza al acceder al detalle del entrenamiento. La aplicación identifica el grupo muscular correspondiente y solicita a la API un máximo de diez ejercicios relacionados con dicho grupo. Para realizar esta comunicación se utiliza Ktor Client, que permite compartir la lógica entre ambas plataformas.

La información recibida de la API se adapta al formato utilizado por la aplicación antes de mostrarse al usuario. Para ello, se seleccionan los nombres disponibles en español, las imágenes correspondientes y se adaptan los nombres de los músculos y del equipamiento. Sin embargo, el número de series, las repeticiones y el tiempo de descanso son definidos por la propia aplicación según el tipo de entrenamiento.

Si la petición no puede completarse o la API no devuelve ejercicios válidos, FitVive utiliza los ejercicios que ya están definidos localmente en la rutina de entrenamiento. De esta forma, el usuario puede consultar o realizar el entrenamiento, aunque la consulta no se haya realizado correctamente.

5.4.2 Asistente virtual

FitVive incorpora un asistente virtual basado en la API de Google Gemini, que permite al usuario realizar consultas relacionadas con el entrenamiento, la alimentación y los hábitos saludables. El objetivo de este asistente es proporcionar respuestas personalizadas teniendo en cuenta la información de cada usuario.

La comunicación con este servicio se realiza mediante Ktor Client. Junto con la consulta realizada por el usuario, la aplicación también envía la información disponible de su perfil, como el nombre, el objetivo, el peso o los días de entrenamiento. Además, se mantiene el historial de conversación para que el asistente pueda responder teniendo en cuenta el contexto.

Antes de enviar la petición, la aplicación construye un mensaje que combina la consulta realizada, la información del perfil del usuario y el historial de la conversación. Una vez recibida la respuesta de la API, el contenido se muestra directamente en la interfaz del chat, permitiendo así mantener una conversación continua con el asistente.

En caso de error o de no ser posible obtener una respuesta válida, la aplicación informa al usuario de que no ha sido posible establecer conexión con el asistente.

5.4.3 Servicio de alimentación

FitVive utiliza la API de Open Food Facts para consultar alimentos y sus valores nutricionales. La información obtenida se utiliza para actualizar el nombre y los principales valores nutricionales de las comidas del plan de alimentación generado por la aplicación. La comunicación con este servicio se realiza mediante Ktor Client, utilizando la misma infraestructura que el resto de los servicios externos, y se inicia al acceder al detalle de un día del plan de alimentación.

En primer lugar, la aplicación muestra el plan generado localmente y calcula el gasto energético diario del usuario a partir de datos como el peso, la altura, el sexo y el objetivo seleccionado. Posteriormente, para cada comida del plan se realiza una consulta a Open Food Facts utilizando el tipo de comida y el objetivo del usuario.

La respuesta del servicio incluye el nombre del producto y sus valores nutricionales por cada cien gramos, concretamente las calorías, las proteínas y los hidratos de carbono. A partir de esta información, la aplicación calcula una porción estimada para cada comida en función de las necesidades energéticas del usuario y adapta los valores nutricionales a dicha cantidad antes de incorporarlos al plan de alimentación.

Antes de realizar una consulta a Open Food Facts, la aplicación comprueba si la información ya se encuentra almacenada en la caché local mediante SQLDelight. Si los datos corresponden al mismo día, se utilizan directamente. En caso contrario, se realiza una nueva consulta a la API y el resultado se guarda en la caché para futuras consultas.

Cuando se obtiene una respuesta válida, se actualizan el nombre, las calorías, las proteínas y los hidratos de carbono de la comida mostrada. El resto de la información definida localmente, como el tipo de comida, los ingredientes, las grasas y si la comida ha sido marcada como completada, se conserva. Si la consulta no devuelve un resultado válido o

se produce algún error, la aplicación mantiene la información del plan de alimentación generado localmente.

5.5 Implementación de las funcionalidades principales

En este apartado se describen las funcionalidades principales implementadas en FitVive. Para cada una de ellas se describe su funcionamiento, las acciones que puede realizar el usuario y la forma en la que la aplicación gestiona la información necesaria para poder llevarlas a cabo.

5.5.1 Gestión de usuarios

La gestión de usuarios permite crear cuentas, iniciar sesión y mantener una sesión activa entre distintas ejecuciones de la aplicación. Además, incorpora un proceso de configuración inicial en el que el usuario introduce la información necesaria para personalizar los planes de entrenamiento y alimentación. Dispone de una pantalla de perfil desde la que es posible consultar y modificar parte de los datos personales.

El registro de nuevos usuarios se realiza mediante un formulario en el que se solicitan el nombre, el correo electrónico y la contraseña. Antes de crear la cuenta, la aplicación valida que todos los datos introducidos sean correctos y que no exista otro usuario registrado con el mismo correo electrónico. Si la información es válida, los datos se almacenan en la base de datos local y el usuario es redirigido a la pantalla de inicio de sesión para acceder a la aplicación con las credenciales creadas.

Durante el inicio de sesión, el usuario introduce su correo electrónico y contraseña. La aplicación verifica las credenciales consultando la información almacenada localmente y, si son correctas, accede a la aplicación con el perfil del usuario y crear una sesión activa. Una vez iniciada la sesión, la aplicación mantiene el usuario autenticado entre las distintas ejecuciones, por lo que no es necesario volver a introducir las credenciales cada vez que se abra la aplicación. Al iniciarse de nuevo, comprueba si existe una sesión almacenada y, en caso afirmativo, se recupera automáticamente el perfil del usuario.

Tras registrarse por primera vez, el usuario debe completar un proceso de configuración inicial dividido en cuatro pasos. En el primero se recopilan datos personales como el sexo, el peso, la altura y la fecha de nacimiento. Posteriormente se selecciona el objetivo principal, pudiendo escoger entre la ganancia de masa muscular, pérdida de peso o mantenimiento. En los dos últimos pasos se indica el número de días de entrenamiento a la semana y el grupo muscular que el usuario desee priorizar.

La información introducida se guarda a medida que el usuario va avanzando por las pantallas. Una vez completados los cuatro pasos, el perfil queda configurado y la aplicación dispone de los datos necesarios para personalizar el plan de entrenamiento y de alimentación del usuario. En la versión de Android, esta información se sincroniza con Firestore para mantener una copia remota del perfil.

Una vez finalizada la configuración inicial, el usuario puede acceder a la pantalla Perfil para consultar su información personal y poder modificar datos como el peso, la altura, el objetivo, el número de días de entrenamiento o el grupo muscular prioritario. Antes de guardar los cambios, la aplicación valida los nuevos valores introducidos y, si son correctos, se actualiza la información almacenada. También es posible cancelar la edición recuperando los datos originales.

Finalmente, desde la pantalla Perfil, el usuario puede cerrar la sesión activa. Esta operación elimina la sesión almacenada y devuelve la aplicación a la pantalla inicial, aunque la información del usuario permanece almacenada localmente para poder volver a acceder a ella posteriormente.

5.5.2 Planes de entrenamiento

La funcionalidad de planes de entrenamiento permite al usuario consultar y realizar las rutinas personalizadas. Estas rutinas están adaptadas a cada perfil, teniendo en cuenta el objetivo establecido, el número de días de días de entrenamiento semanales y el grupo muscular prioritario.

Al acceder a Entrenamientos, la aplicación muestra los distintos días disponibles del plan semanal. Para cada día, la rutina se genera a partir de la configuración del perfil del usuario, permitiendo acceder al detalle de cada entrenamiento. Las rutinas no se almacenan en la base de datos, sino que la aplicación las genera automáticamente al acceder a cada entrenamiento. Una vez finalizado el entrenamiento, la aplicación registra la sesión realizada en la base de datos local mediante SQLDelight para conservar el historial del usuario.

La distribución de las rutinas varía en función de los días de entrenamiento seleccionados por el usuario en la configuración del perfil. Al acceder al detalle de un entrenamiento, la aplicación consulta la API de ejercicios para completar la información de cada ejercicio, como el músculo trabajado, el equipamiento necesario o las imágenes asociadas. Si la consulta no devuelve resultados o se produce algún error, se utiliza la información generada localmente.

Durante la realización del entrenamiento, la aplicación muestra cada ejercicio junto con el número de series y repeticiones recomendadas. El usuario puede registrar el peso utilizado, el número de repeticiones realizadas y marcar cada serie como completada, pudiendo modificar esta información durante el entrenamiento.

Una vez completado el entrenamiento, la aplicación registra la fecha, el día del plan y el grupo muscular trabajado en la base de datos local. En la versión de Android, esta información se sincroniza, además, con Firestore, donde se almacena el detalle de cada ejercicio realizado. Finalmente, el usuario vuelve a la pantalla de Entrenamientos, donde puede seleccionar otra sesión del plan.

5.5.3 Planificación de la dieta

La funcionalidad de planificación de la dieta permite al usuario consultar un menú semanal adaptado a su objetivo. Desde la pantalla principal se puede acceder a la sección

Dieta, donde se muestran los diferentes días de la semana. De lunes a sábado existe un plan de alimentación, mientras que el domingo se presenta como día libre sin un menú estricto.

Al seleccionar un día, la aplicación genera un plan el cual contiene desayuno, comida, merienda y cena. Este plan se adapta al objetivo del usuario, distinguiendo entre ganancia de masa muscular, pérdida de peso y mantenimiento. Las comidas incluyen información como el nombre, los ingredientes, las calorías y la cantidad de macronutrientes. El plan semanal no se almacena en la base de datos, sino que la aplicación es la que define el plan.

Además, la aplicación calcula las necesidades calóricas diarias del usuario utilizando su peso, altura, sexo y objetivo. Este valor se utiliza para establecer el objetivo diario de calorías y para distribuir las calorías entre las distintas comidas del día, ajustando también sus macronutrientes.

Una vez generado el plan personalizado, la aplicación consulta Open Food Facts para completar la información nutricional de las comidas. La cantidad del alimento se adapta a las calorías asignadas a cada comida. Si la consulta no devuelve datos o se produce algún error, se muestra la planificación almacenada en la base de datos local, garantizando que el usuario pueda consultar su dieta igualmente.

Los resultados obtenidos de Open Food Facts se almacenan temporalmente en la base de datos local para evitar realizar las mismas consultas durante el mismo día. Esta información se organiza según el objetivo, el tipo de comida y el objetivo calórico.

El usuario también puede marcar la comida como completada o desmarcarla. Este estado se guarda en la base de datos local junto con el día, el objetivo y el tipo de comida y, en Android, se sincroniza con Firestore para mantener el progreso del usuario. Al volver a acceder al mismo día, la aplicación recupera las comidas completadas previamente. El botón Finalizar permite regresar a la pantalla Dieta. La aplicación no registra la finalización de un día completo, sino el estado de cada comida de forma individual.

Esta funcionalidad tiene la misma implementación en ambas plataformas. El plan de alimentación se genera de forma local y no se almacena en Firestore, ya que puede generarse de nuevo a partir de la información del usuario. En cambio, el estado de las comidas completadas sí se sincroniza con Firestore en Android, al tratarse de información que representa el progreso del usuario y no puede regenerarse. De este modo, se reduce la cantidad de datos sincronizados y la aplicación puede seguir funcionando con la información almacenada localmente cuando no hay conexión. En iOS, esta información se almacena únicamente de forma local.

5.5.4 Registro de progreso

La aplicación incorpora una funcionalidad de registro de progreso a la que se puede acceder desde la pantalla principal. En esta sección, el usuario consulta la evolución de su peso y el historial de entrenamientos. Además, se muestran el peso actual, los días de

entrenamiento consecutivos, el grupo muscular prioritario y un resumen de la evolución del peso mediante una gráfica y la posibilidad de registrar un nuevo peso.

El usuario puede registrar periódicamente un peso junto con una valoración semanal y unas notas opcionales. Antes de guardar esta información, la aplicación comprueba que el peso introducido sea un número válido comprendido entre 30 y 300 kg. Una vez validado el peso, el registro se almacena junto a la fecha, la valoración, las notas y el peso actualizado. Tras guardar el registro, la aplicación muestra un mensaje personalizado que analiza la evolución del peso en función del objetivo establecido por el usuario, como se observa en la Figura 22.



Figura 22. Registro de peso y mensaje del análisis informativo. Fuente: Elaboración propia.

La evolución del peso se representa mediante una gráfica construida a partir del historial de registros almacenados. Además, la pantalla muestra el peso inicial, el peso actual y la diferencia entre ambos, permitiendo consultar el historial completo y editar o eliminar los registros existentes, como se muestra en la Figura 23.

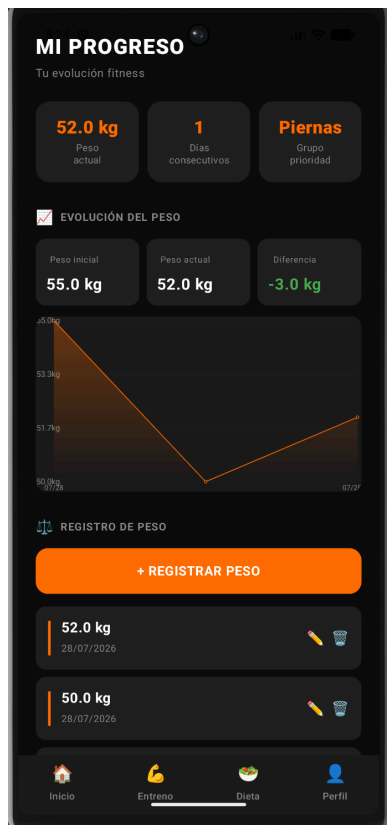


Figura 23. Pantalla de progreso con la gráfica de evolución de peso. Fuente: Elaboración propia.

Además de la representación gráfica, la aplicación conserva un historial de todos los registros de peso realizados por el usuario. Cada registro muestra la fecha, el peso, la valoración semanal y las notas asociadas, pudiendo editar o eliminar la información.

La información del progreso se almacena en la base de datos local, utilizada para construir el historial y la gráfica. En Android, los nuevos registros de peso se sincronizan con Firestore para conservar el progreso del usuario, mientras que en iOS esta información permanece únicamente en la base de datos local.

El almacenamiento local permite que esta funcionalidad continúe disponible cuando no hay conexión a Internet. Al volver a acceder a la sección de progreso, la aplicación recupera automáticamente los datos almacenados y los muestra al usuario.

6 Resultados y validación

En este capítulo se presentan los resultados obtenidos durante el desarrollo de FitVive y se comprueba el correcto funcionamiento de las principales funcionalidades. Además, se describen las pruebas realizadas para verificar su funcionamiento, una evaluación del resultado final y las principales limitaciones de la aplicación.

6.1 Vistas definitivas de la aplicación

En este apartado se presentan las vistas finales de FitVive, mostrando las principales pantallas desarrolladas durante el proyecto. Cada una de ellas va acompañada de una breve descripción de su funcionalidad con el objetivo de mostrar el resultado final de la aplicación y las características implementadas.

6.1.1 Pantalla de bienvenida

La pantalla de bienvenida es el punto de acceso a FitVive y es la primera vista que se muestra al iniciar la aplicación. Se ha diseñado con una interfaz sencilla y minimalista, utilizando un fondo oscuro combinado con tonos naranjas para destacar los elementos interactivos.

En la parte superior se muestra el logotipo, junto con el eslogan “Tu rutina. Tu progreso.”, mientras que en la parte inferior se encuentran las opciones Iniciar sesión y Crear cuenta, desde las que el usuario puede iniciar sesión o crear una cuenta nueva para acceder a la aplicación.

La Figura 24 muestra la pantalla de bienvenida de FitVive en Android e iOS. Puede observarse que la distribución de los elementos y el diseño visual se mantienen prácticamente idénticos en ambas plataformas, garantizando una experiencia de usuario uniforme.

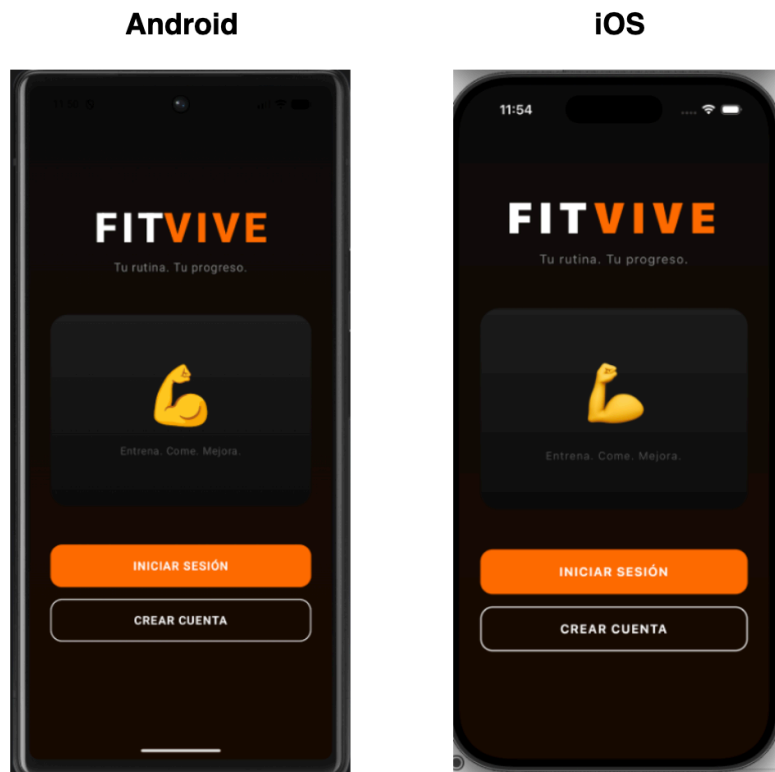


Figura 24. Pantalla de bienvenida de FitVive en Android e iOS. Fuente: Elaboración propia.

6.1.2 Registro e inicio de sesión

Las pantallas de registro e inicio de sesión permiten al usuario autenticarse para acceder a las funcionalidades de la aplicación. Ambas mantienen el mismo diseño que la pantalla de bienvenida, utilizando un fondo oscuro y detalles en color naranja para ofrecer una apariencia uniforme.

La pantalla de inicio de sesión incluye los campos necesarios para introducir el correo electrónico y la contraseña, además del botón Iniciar sesión, que permite acceder a la aplicación. También dispone de un enlace que dirige al usuario a la pantalla de registro en caso de que todavía no tenga una cuenta.

Por otra parte, la pantalla de registro permite crear una cuenta nueva introduciendo el nombre de usuario, el correo electrónico y la contraseña. Una vez completado el registro, el usuario accede a la configuración inicial de la aplicación, donde se recopila toda la información necesaria del usuario para generar los planes personalizados de alimentación y entrenamiento.

La Figura 25 muestra las pantallas de inicio de sesión y registro. En ellas pueden observarse los formularios utilizados para la autenticación y el registro de nuevos usuarios, manteniendo la línea de diseño del resto de la aplicación.

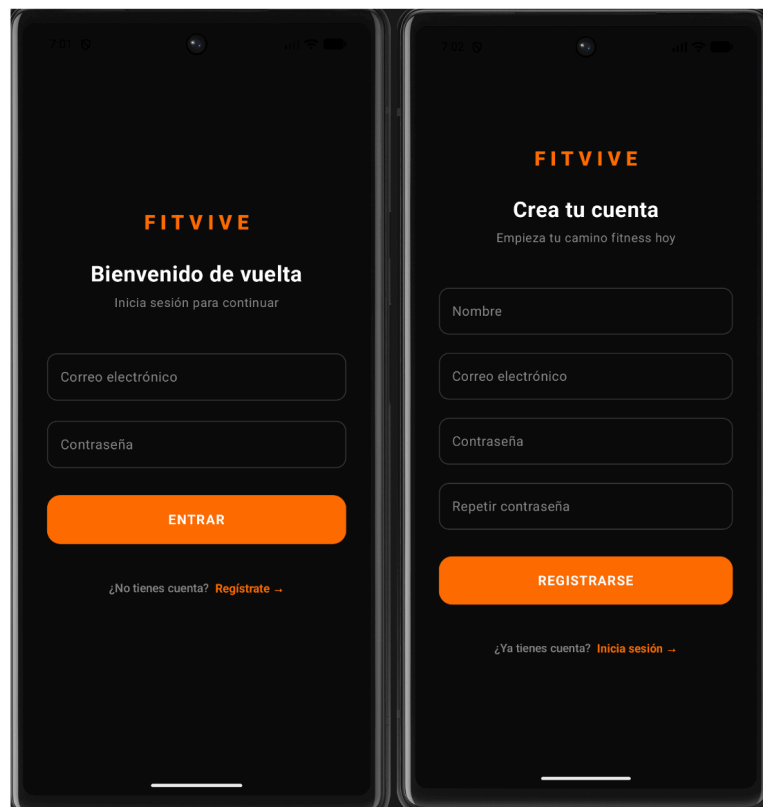


Figura 25. Pantallas de inicio de sesión y registro. Fuente: Elaboración propia.

6.1.3 Configuración inicial

La configuración inicial se muestra automáticamente después de que el usuario complete el proceso de registro. Su objetivo es recopilar la información necesaria para personalizar el funcionamiento de la aplicación y generar los planes de entrenamiento y alimentación.

El proceso está dividido en cuatro pantallas consecutivas para facilitar la introducción de los datos. En ellas, el usuario debe indicar su información personal, como el sexo, el peso, la altura y la fecha de nacimiento, así como el objetivo físico, el número de días de entrenamientos semanales y el grupo muscular al que desea dar prioridad.

Una vez completados todos los pasos, la información queda almacenada en el perfil del usuario y la aplicación genera automáticamente los planes personalizados.

La Figura 26 muestra las distintas pantallas que componen el proceso de configuración inicial.

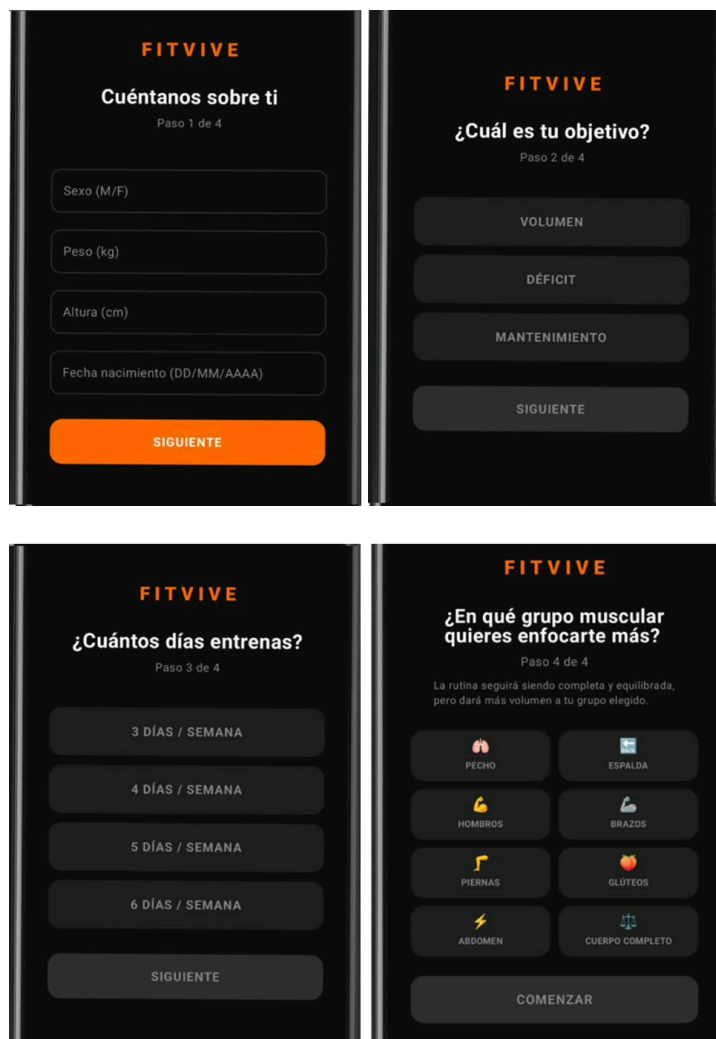


Figura 26. Proceso de configuración inicial de FitVive. Fuente: Elaboración propia.

6.1.4 Pantalla principal

La pantalla principal es la vista que se muestra al usuario una vez completada la configuración inicial o después de iniciar sesión. Desde ella se puede acceder de forma rápida a las funcionalidades de FitVive.

En la parte superior se muestra un saludo personalizado junto con el mensaje motivacional “*No pain, no gain*”, una expresión utilizada en el ámbito del deporte que hace referencia a la importancia del esfuerzo y la constancia para alcanzar los objetivos deportivos. Además, se incluyen el logotipo de la aplicación, una ilustración relacionada con el entrenamiento y un indicador con el número de días de entrenamiento semanales configurados por el usuario.

La parte central de la pantalla está formada por cuatro bloques que permiten acceder a las secciones principales de la aplicación: Entrenamientos, Dieta personalizada, Progreso y Asistente IA. Cada uno de ellos incorpora un icono y una breve descripción para facilitar su identificación.

En la parte inferior se encuentra una barra de navegación que permite acceder rápidamente a las secciones de Inicio, Entreno, Dieta y Perfil.

La Figura 27 muestra la pantalla principal de FitVive en Android e iOS. Puede observarse que la distribución de los elementos y el diseño visual se mantienen idénticos en ambas plataformas.

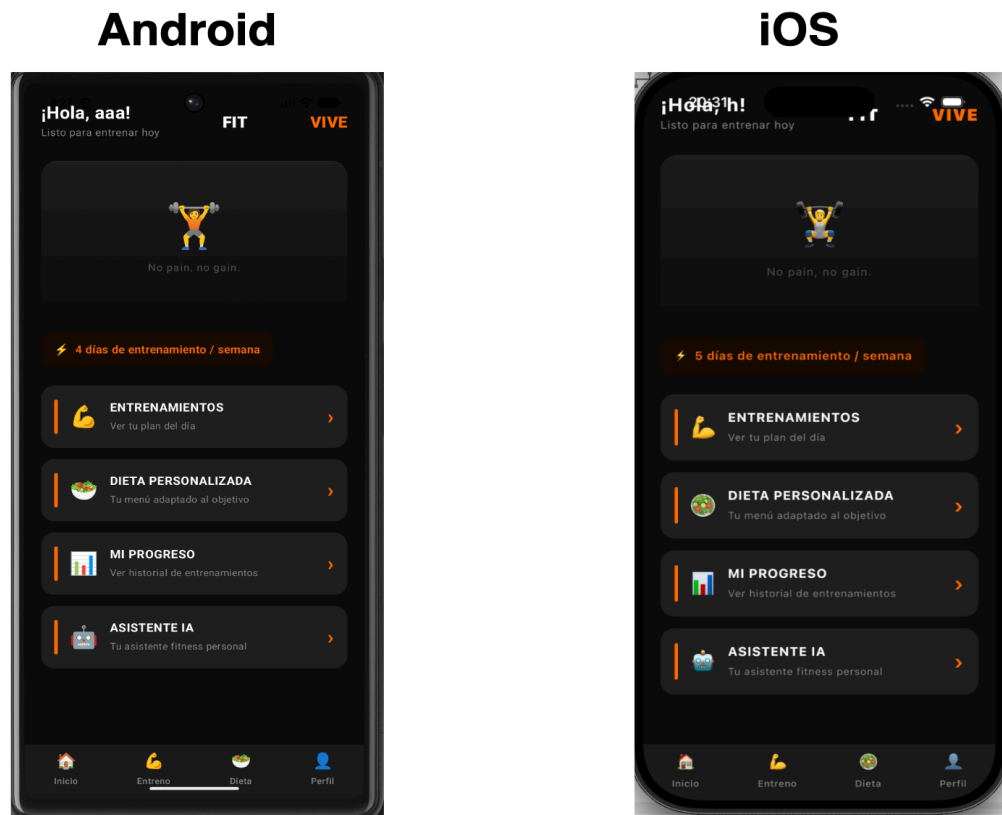


Figura 27. Pantalla principal de FitVive en Android e iOS. Fuente: Elaboración propia.

6.1.5 Plan de entrenamiento

El plan de entrenamiento permite al usuario consultar la planificación generada por la aplicación y registrar la realización de cada sesión. Desde esta sección puede seleccionar cualquiera de los días de entrenamiento disponibles para acceder a la rutina correspondiente.

En la pantalla de detalle se muestran los ejercicios que componen la rutina, junto con información como el número de series, las repeticiones recomendadas y el tiempo de descanso entre ellas. Además, el usuario puede registrar el peso utilizado y el número de repeticiones realizadas en cada serie, así como marcar cada una de ellas como completada.

La planificación se adapta a la información proporcionada durante la configuración inicial, teniendo en cuenta el objetivo físico, el número de días de entrenamiento semanales y el grupo muscular prioritario seleccionado.

La Figura 28 muestra la pantalla principal de entrenamientos, en la que el usuario puede seleccionar el día correspondiente a la rutina que desea realizar.

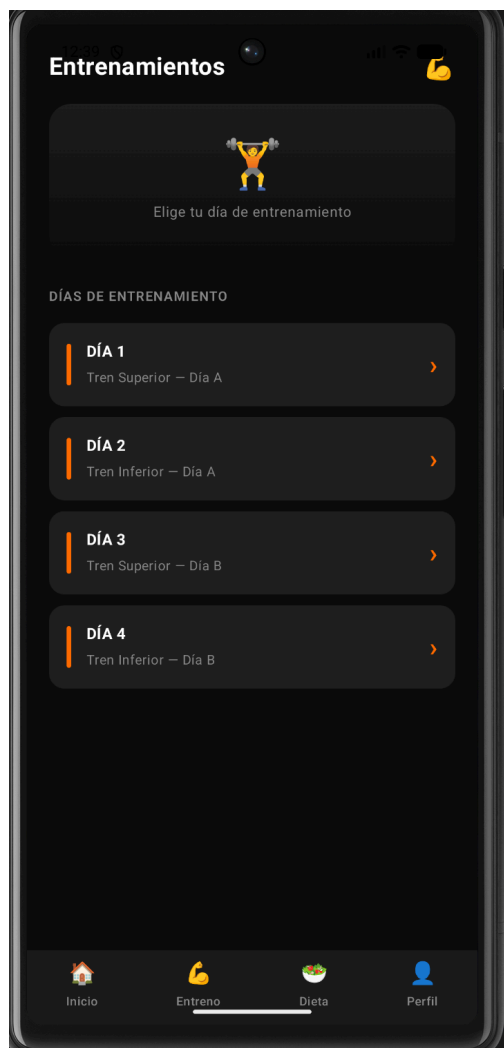


Figura 28. Pantalla principal de entrenamientos. Fuente: Elaboración propia.

Al seleccionar uno de los días disponibles, se accede al detalle del entrenamiento, donde se muestran los ejercicios de la rutina y los campos necesarios para registrar el peso y las repeticiones realizadas en cada serie. La Figura 29 muestra esa pantalla.

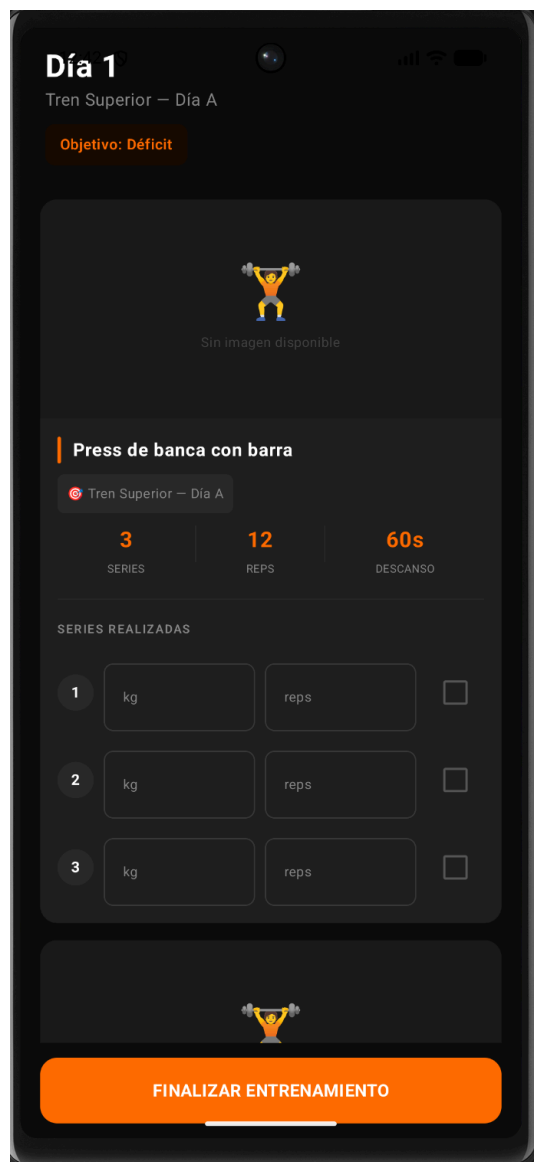


Figura 29. Detalle de un entrenamiento. Fuente: Elaboración propia.

6.1.6 Planificación de la dieta

La planificación de la dieta permite al usuario consultar el plan de alimentación personalizado generado por la aplicación. Desde esta sección puede seleccionar cualquiera de los días de la semana para acceder al menú correspondiente.

En la pantalla de detalle se muestran las comidas planificadas para el día seleccionado, organizadas en desayuno, comida, merienda y cena. Para cada una de ellas se indica el nombre del plato, los ingredientes, las cantidades recomendadas y la información nutricional correspondiente. Además, el usuario puede marcar cada comida como completada para realizar un seguimiento de su cumplimiento.

El plan de alimentación se adapta al objetivo físico establecido durante la configuración inicial, ofreciendo una planificación acorde con las necesidades del usuario.

La Figura 30 muestra la pantalla principal de la planificación de la dieta, donde el usuario puede seleccionar el día cuya planificación desea consultar.

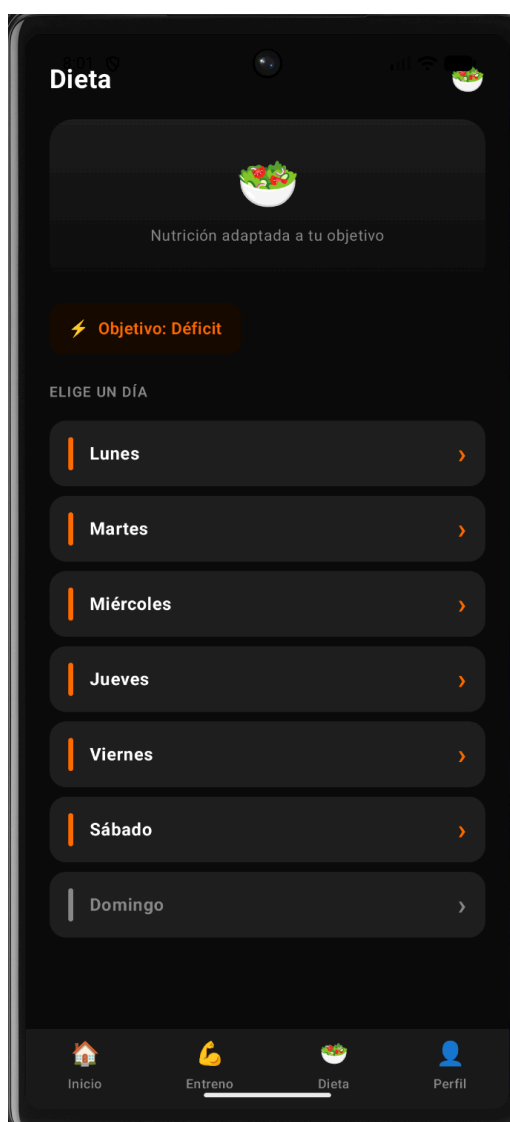


Figura 30. Pantalla principal de la planificación de la dieta. Fuente: Elaboración propia.

Al seleccionar uno de los días disponibles, se accede al detalle del menú correspondiente, donde se muestran las comidas planificadas y la información asociada a cada una de ellas. La Figura 31 muestra esta pantalla.

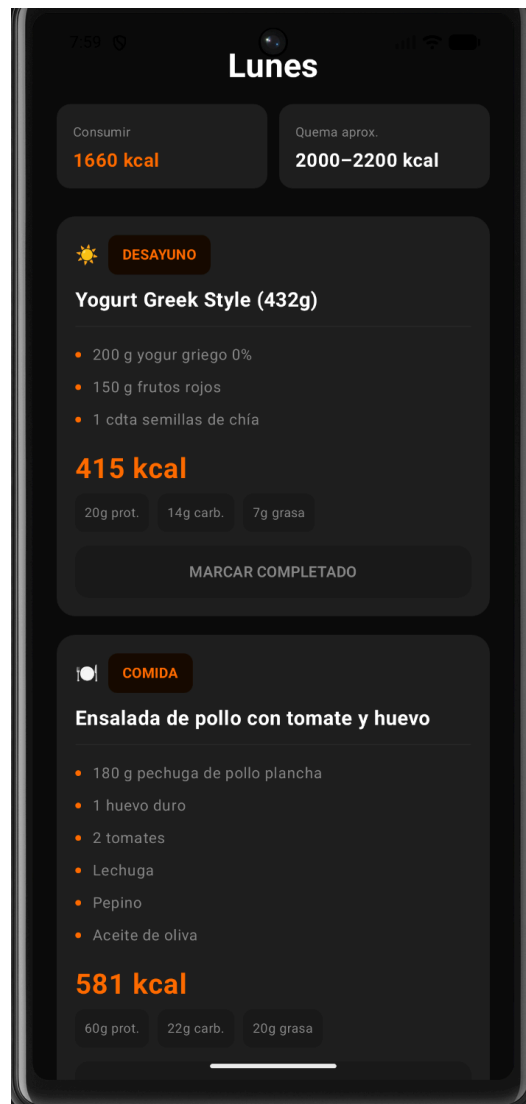


Figura 31. Detalle de la planificación de la dieta. Fuente: Elaboración propia.

6.1.7 Seguimiento del progreso

La sección de seguimiento de progreso permite al usuario consultar la evolución de su peso y realizar un seguimiento. En la pantalla principal de esta sección se muestra un resumen con información relevante, como el peso actual, el número de días consecutivos de entrenamiento y el grupo muscular prioritario configurado.

Además, se muestra una gráfica que representa la evolución del peso a lo largo del tiempo, junto con un historial de los registros almacenados.

La Figura 32 muestra la pantalla principal del progreso, donde pueden observarse el resumen de información, la gráfica de evolución y el historial de registros.

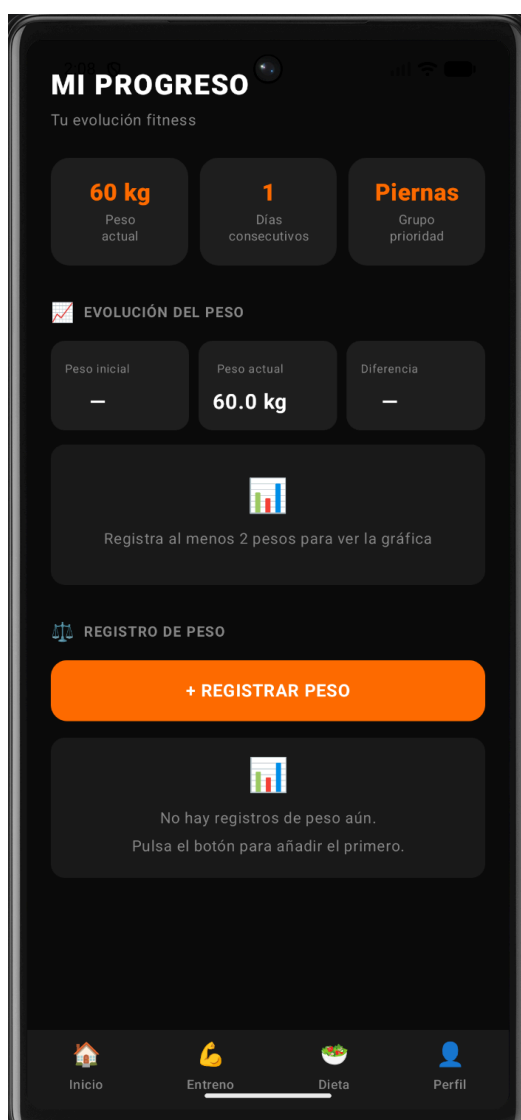


Figura 32. Pantalla principal del seguimiento del progreso. Fuente: Elaboración propia.

Para registrar un nuevo peso, la aplicación dispone de una pantalla específica en la que el usuario puede introducir su peso actual, valorar la evolución de la semana y añadir observaciones. Una vez guardado el registro, la información queda actualizada y pasa a formar parte del historial de progreso del usuario.

La Figura 33 muestra la pantalla utilizada para registrar un nuevo peso.

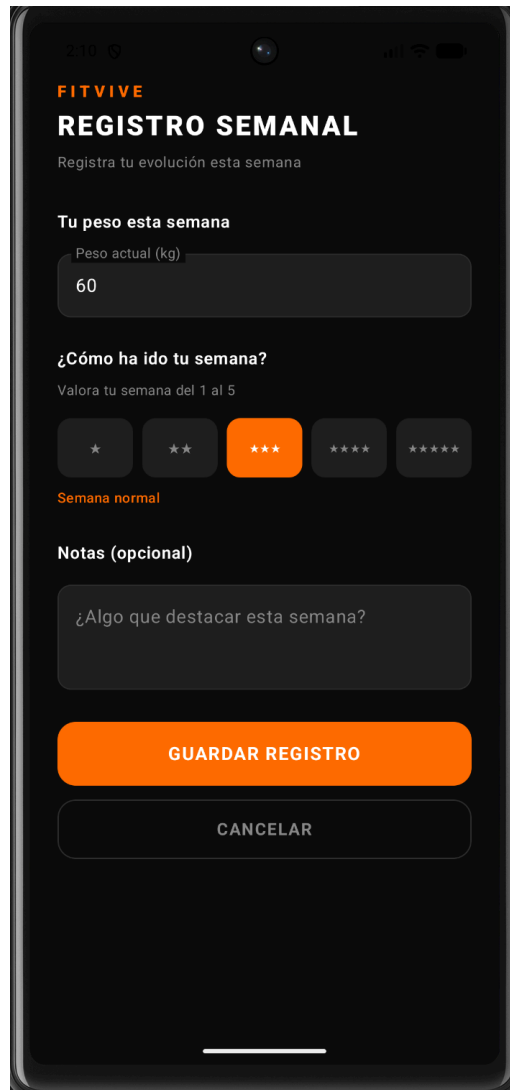


Figura 33. Registro de un nuevo peso. Fuente: Elaboración propia.

6.1.8 Asistente de IA

El asistente IA permite al usuario realizar consultas sobre el entrenamiento, alimentación y hábitos saludables a través de un chat. Su funcionamiento se basa en un sistema de conversación mediante mensajes.

El usuario puede escribir preguntas de forma libre y recibir respuestas generadas por el asistente IA relacionadas con aspectos como la actividad física o la alimentación. De esta forma, puede solicitar información sobre ejercicios, recomendaciones nutricionales, rutinas de entrenamiento o cualquier otra consulta relacionada con la actividad física.

La interfaz está organizada en un historial de conversación donde se muestran los mensajes enviados por el usuario y las respuestas generadas por el asistente, permitiendo mantener un seguimiento de la conversación.

La Figura 34 muestra la pantalla del Asistente IA, donde puede observarse el chat utilizado para realizar consultas y recibir respuestas.



Figura 34. Pantalla del Asistente IA. Fuente: Elaboración propia.

6.1.9 Perfil y configuración

La pantalla de perfil permite al usuario consultar y gestionar su información personal. En ella se muestran los principales datos configurados durante el registro, como el nombre del usuario, el correo electrónico y la información utilizada para personalizar los planes de entrenamiento y alimentación.

Además, desde esta sección el usuario puede modificar estos datos y actualizar la información del perfil cuando sea necesario. También dispone de la opción de cerrar sesión.

La Figura 35 muestra la pantalla principal del perfil, donde puede observarse los datos del usuario.



Figura 35. Pantalla principal del perfil. Fuente: Elaboración propia.

La Figura 36 muestra la pantalla que corresponde a la edición del perfil.

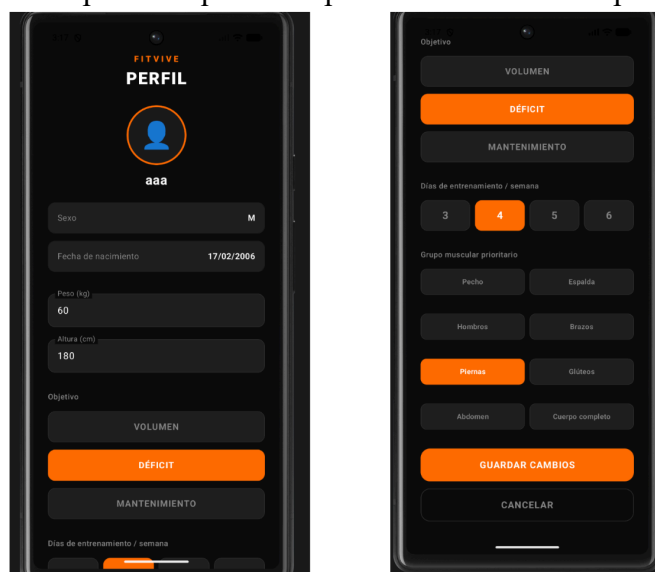


Figura 36. Pantalla de edición del perfil. Fuente: Elaboración propia.

Además, esta sección ofrece la posibilidad de configurar las notificaciones de la aplicación, activando o desactivando los recordatorios de entrenamiento y de la planificación de la dieta, pudiendo ajustar el horario. En la Figura 37 se muestra la pantalla de configuración de notificaciones.

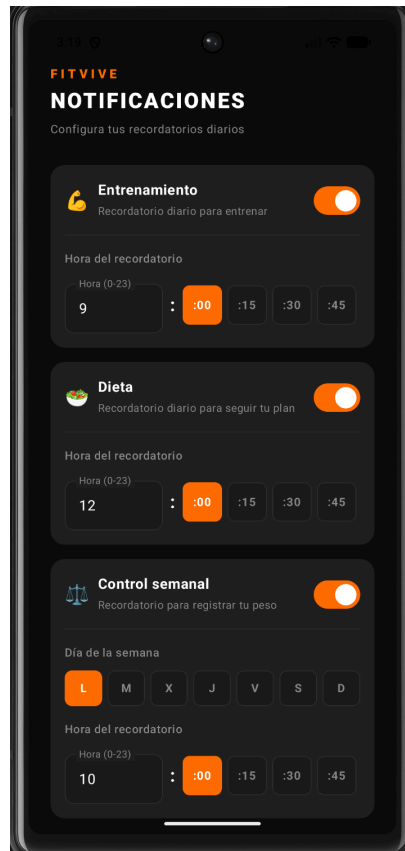


Figura 37. Pantalla de configuración de notificaciones. Fuente: Elaboración propia.

6.2 Pruebas realizadas

Con el objetivo de comprobar el correcto funcionamiento de FitVive, se han realizado diferentes pruebas funcionales sobre las principales características de la aplicación. Estas pruebas han permitido verificar el correcto funcionamiento de las funcionalidades implementadas.

Las comprobaciones se han realizado utilizando tanto datos válidos como valores incorrectos, con el fin de verificar que la aplicación mostrará los correspondientes mensajes de error. También se ha comprobado el comportamiento de la aplicación frente a posibles fallos de conexión.

6.2.1 Pruebas funcionales

Se han realizado pruebas funcionales sobre las principales características de FitVive con el objetivo de verificar su correcto funcionamiento. Estas pruebas han permitido comprobar que la aplicación respondía de forma adecuada en cada caso.

La Tabla 12 muestra un resumen de las principales pruebas funcionales realizadas y los resultados obtenidos.

Funcionalidad	Descripción de la prueba	Resultado
Registro de usuario	Crear una cuenta con datos válidos	Superada
Validación del registro	Introducir datos incorrectos durante el registro	Superada
Inicio de sesión	Acceder con credenciales válidas	Superada
Configuración inicial	Configurar el perfil del usuario	Superada
Plan de entrenamiento	Generar una rutina personalizada	Superada
Registro de entrenamiento	Registrar series repeticiones y peso	Superada
Planificación de la dieta	Generar una planificación de dieta adaptada al usuario	Superada
Seguimiento del progreso	Registrar y visualizar la evolución del peso	Superada
Asistente IA	Enviar una consulta y recibir respuesta	Superada
Edición del perfil	Modificar los datos personales	Superada
Configuración de notificaciones	Activar y modificar recordatorios	Superada
Cierre de sesión	Finalizar la sesión del usuario	Superada

Tabla 13. Resumen de las pruebas funcionales realizadas. Fuente: Elaboración propia.

Los resultados obtenidos han confirmado el correcto funcionamiento de las principales funcionalidades implementadas en la aplicación. Además, se ha verificado que la aplicación respondía correctamente tanto en condiciones normales como ante errores de validación y problemas de conexión.

6.2.2 Pruebas multiplataforma

Dado que FitVive ha sido desarrollada mediante Kotlin Multiplatform, se han realizado pruebas en dispositivos Android e iOS con el objetivo de comprobar su correcto funcionamiento en ambas plataformas.

Durante las pruebas se ha verificado la navegación entre pantallas, la visualización de la interfaz, el acceso a las diferentes funcionalidades, la persistencia de los datos y la comunicación con los servicios externos. Los resultados obtenidos han sido prácticamente idénticos en ambas plataformas, garantizando una experiencia de usuario uniforme.

6.3 Evaluación

Una vez finalizado el desarrollo y realizadas las pruebas funcionales, se realiza una valoración del resultado obtenido. En este apartado se analiza si se han cumplido los objetivos planteados al inicio del proyecto y se presenta una valoración del trabajo realizado.

6.3.1 Cumplimiento de los objetivos

El principal objetivo de este proyecto era desarrollar una aplicación móvil multiplataforma que ayudara a los usuarios a organizar sus entrenamientos, planificar su alimentación y realizar un seguimiento de su progreso. Este objetivo se ha alcanzado con el desarrollo de FitVive, una aplicación disponible para Android e iOS.

Asimismo, se han implementado las funcionalidades previstas, como la generación personalizada de entrenamiento y alimentación, el seguimiento del progreso, el asistente basado en inteligencia artificial y el sistema de notificaciones, cumpliendo así los objetivos definidos para el proyecto.

6.3.2 Valoración del resultado

La aplicación ofrece una interfaz sencilla e intuitiva, facilitando la navegación entre las distintas pantallas. Además, el uso de Kotlin Multiplatform ha permitido reutilizar gran parte del código en Android e iOS, reduciendo el tiempo de desarrollo y manteniendo un funcionamiento similar en ambas plataformas.

El resultado obtenido es una aplicación funcional y estable que cumple con los objetivos planteados y ofrece una herramienta útil para el seguimiento de hábitos saludables.

En definitiva, el proyecto ha permitido desarrollar una aplicación que cumple con los objetivos planteados y que puede servir como base para futuras ampliaciones y mejoras.

6.4 Limitaciones

Aunque se han alcanzado los objetivos principales del proyecto, la aplicación presenta algunas limitaciones que podrían corregirse en futuras versiones.

En primer lugar, las cuentas de usuario se gestionan de forma local, por lo que un usuario registrado en un dispositivo debe volver a registrarse si utiliza otro diferente.

Además, la sincronización con Firestore solo está disponible en Android y únicamente se aplica a determinados datos, por lo que no existe una sincronización completa entre dispositivos.

Otra limitación es la dependencia de servicios externos. Tanto el asistente IA como las APIs utilizadas para obtener información sobre ejercicios y alimentación requieren conexión a internet. Si alguno de los servicios no está disponible, algunas funcionalidades pueden verse limitadas, aunque la aplicación utiliza información local cuando es posible.

El cálculo de las necesidades calóricas puede mejorarse. Actualmente se utiliza el peso, la altura, el sexo y el objetivo del usuario, pero no se tiene en cuenta la edad. Por ello, los resultados obtenidos deben considerarse una estimación y podrían mejorarse incorporando más variables relacionadas con el perfil del usuario.

Por último, la personalización de los planes de entrenamientos y alimentación también puede mejorarse incorporando parámetros, como el nivel de experiencia, la existencia de

lesiones o las preferencias alimentarias, permitiendo generar recomendaciones todavía más adaptadas a cada usuario.

Estas limitaciones no impiden el funcionamiento de FitVive, pero representan posibles mejoras que podrían incorporarse en futuras versiones de la aplicación y ofrecer una experiencia de usuario más completa.

7 Conclusiones y trabajos futuros

En este capítulo se presentan las conclusiones obtenidas tras el desarrollo de FitVive, realizando una valoración del trabajo realizado y del grado de cumplimiento de los objetivos planteados. Además, se proponen posibles mejoras y nuevas funcionalidades que podrían incorporarse en futuras versiones de la aplicación.

7.1 Conclusiones

El objetivo principal de este Trabajo de Fin de Grado era desarrollar una aplicación móvil multiplataforma que ayudara a los usuarios a organizar sus entrenamientos, planificar su alimentación y realizar un seguimiento de su progreso. Este objetivo se ha alcanzado mediante el desarrollo de FitVive, una aplicación disponible para Android e iOS.

Durante el desarrollo se han implementado las principales funcionalidades previstas, como la generación personalizada de planes de entrenamiento y alimentación, el seguimiento del progreso, el asistente basado en inteligencia artificial y el sistema de notificaciones.

Además, el uso de Kotlin Multiplatform ha permitido reutilizar gran parte del código entre ambas plataformas, facilitando el desarrollo y manteniendo un funcionamiento similar en Android e iOS.

En conjunto, el resultado obtenido es una aplicación funcional que cumple con los objetivos planteados y ofrece una herramienta útil para ayudar a los usuarios a mejorar sus hábitos de entrenamiento y alimentación.

Además, este proyecto me ha permitido poner en práctica los conocimientos adquiridos durante el grado, especialmente en el desarrollo de aplicaciones móviles, el uso de bases de datos, el consumo de APIs y el desarrollo multiplataforma.

7.2 Trabajos futuros

Aunque FitVive cumple con los objetivos planteados, existen ciertos aspectos que podrían mejorarse o ampliarse en futuras versiones con el fin de ofrecer una experiencia de usuario más completa y personalizada.

- Mejorar el proceso de autenticación, incorporando un sistema que permita utilizar la misma cuenta desde diferentes dispositivos y un proceso de registro más completo.
- Añadir soporte para varios idiomas, facilitando el uso de la aplicación por usuarios de diferentes países.
- Mejorar la personalización de los planes de entrenamiento y dieta, teniendo en cuenta más información del usuario, como su nivel de experiencia, la existencia de posibles lesiones, la edad o sus preferencias alimentarias.

- Mejorar la sincronización de datos, permitiendo realizar una copia de seguridad completa de la información del usuario desde ambas plataformas.
- Mejorar el asistente basado en inteligencia artificial, ofreciendo respuestas más personalizadas y permitiendo escanear productos para identificar sus valores nutricionales y proporcionar información relevante al usuario.

8 Bibliografía

- [1] Organización Mundial de la Salud. (2024). Actividad física. <https://www.who.int/es/news-room/fact-sheets/detail/physical-activity>
- [2] Apple. (s.f.) App Store: *Salud y forma física*. <https://apps.apple.com/es/iphone/grouping/171904>
- [3] MyFitnessPal. (s.f.) *What are the features of MyFitnessPal Premium?* https://support.myfitnesspal.com/hc/en-us/articles/360032625951-What-are-the-features-of-MyFitnessPal-Premium?utm_source
- [4] Fitbod. (s.f.). *Fitbod*. <https://www.fitbod.me/>
- [5] Freeletics. (s.f.). *Freeletics*. <https://www.freeletics.com/es/>
- [6] Nike. (s.f.). *Nike Training Club*. <https://www.nike.com/es/ntc-app>
- [7] Android Developers. (s.f.) *Kotlin and Android*. <https://developer.android.com/kotlin>
- [8] Apple. (s. f.). *Swift*. *Apple Developer*. <https://developer.apple.com/swift/>
- [9] JetBrains. (s. f.). *Kotlin Multiplatform*. <https://kotlinlang.org/multiplatform/>
- [10] JetBrains (s.f). *Compose Multiplatform*. <https://www.jetbrains.com/compose-multiplatform/>
- [11] Firebase. (s.f). *Cloud Firestore*. <https://firebase.google.com/docs/firestore>
- [12] Ktor. (s.f.). *Ktor Framework*. <https://ktor.io/>
- [13] Google AI. (s.f.). *Gemini API Documentation*. <https://ai.google.dev/gemini-api/docs>
- [15] Wger Project. (s.f.). *Wger REST API*. <https://wger.de/en/software/api>
- [16] Open Food Facts. (s.f.). *Open Food Facts API*. <https://world.openfoodfacts.org/data>
- [17] Touchlab. (s.f.). *SQLDelight*. <https://sqldelight.github.io/sqldelight/>
- [18] draw.io. (s.f.) draw.io <https://www.drawio.com/>
- [19] Google. (s.f.). *Android Studio*. <https://developer.android.com/studio>
- [20] PCMag UK. (2023). *MyFitnessPal - Review 2023*. <https://uk.pcmag.com/iphone-apps/5599/myfitnesspal>

[21] Patch. (2020). Fitbod workout app: personalized approach to getting fit in 2020. <https://patch.com/illinois/chicago/fitbod-workout-app-personalized-approach-getting-fit-2020>

[22] HomexGym. (2026). Ranking 2026 Apps Fitness >> Las mejores Aplicaciones para hacer Ejercicio en Casa. <https://homexgym.com/blog/ranking-mejores-apps-ejercicio-en-casa/>

[23] Trailrun. (s.f). Strava lanza la integración de Nike Run Club y Nike Training Club. https://www.trailrun.es/actualidad/strava-lanza-integracion-nike-run-club-nike-training-club_282997_102.html

[24] Shakers. (s.f). ¿Cuánto cuesta contratar a un programador Freelance? <https://blog.shakersworks.com/precio-contratar-programador-freelance>

9 Anexos

En este capítulo se incluyen distintos anexos relacionados con el desarrollo de FitVive. Entre ellos se encuentran el manual de usuario, el manual del programador y otros aspectos relacionados con la configuración y el funcionamiento de FitVive.

9.1 Manual de usuario

El presente manual tiene como objetivo explicar el funcionamiento básico de FitVive desde el punto de vista del usuario.

1. Registro e inicio de sesión

1.1 Crear una cuenta

1. Abrir la aplicación.
2. Seleccionar Crear cuenta.
3. Introducir el nombre, el correo electrónico y la contraseña.
4. Pulsar el botón Registrarse.
5. Una vez completado el registro, acceder a la pantalla inicio de sesión.

1.2 Iniciar sesión

1. Seleccionar Iniciar sesión.
2. Introducir el correo y la contraseña.
3. Pulsar Iniciar sesión para acceder a la aplicación.

Nota: Visualizar la Figura 25.

2. Configuración inicial.

1. Introducir los datos personales (sexo, peso, altura y fecha de nacimiento).
2. Seleccionar el objetivo físico.
3. Elegir el número de días de entrenamiento por semana (de 3 a 6 días).
4. Seleccionar el grupo muscular prioritario.
5. Pulsar el botón Comenzar para finalizar la configuración inicial.

Nota: Visualizar la Figura 26.

3. Plan de entrenamiento.

3.1 Consultar un entrenamiento.

1. Acceder a la sección Entrenamientos.
2. Seleccionar el día de entrenamiento deseado.
3. Consultar los ejercicios de la rutina.

3.2 Registrar un entrenamiento

1. Introducir el peso utilizado y las repeticiones realizadas en cada serie.
2. Marcar las series completadas.
3. Pulsar Finalizar entrenamiento para guardar la sesión.

Nota: Visualizar las Figuras 28 y 29.

4. Planificación de la dieta

4.1 Consultar la dieta

1. Acceder a la sección Dieta.
2. Seleccionar el día correspondiente.
3. Consultar las comidas propuestas.

4.2 Registrar una comida

1. Pulsar sobre la comida realizada para marcarla como completada.
2. Volver a pulsarla si se desea deshacer el registro.

Nota: Visualizar las Figuras 30 y 31.

5. Seguimiento del progreso

1. Acceder a la sección Progreso
2. Registrar un nuevo peso.
3. Introducir la valoración semanal y las observaciones, si se desea.
4. Consultar las estadísticas mostradas por la aplicación.

Nota: Visualizar las Figuras 32 y 33.

6. Asistente IA

1. Acceder a la sección Asistente IA.
2. Escribir una consulta relacionada con el entrenamiento o la alimentación.
3. Pulsar el botón de envío.
4. Consultar la respuesta generada por el asistente.

Nota: Visualizar la Figura 34.

7. Perfil y configuración

7.1 Editar el perfil

1. Acceder a Perfil.
2. Pulsar Editar perfil.
3. Modificar los datos necesarios.
4. Guardar los cambios.

7.2 Configurar las notificaciones

1. Acceder a la configuración de notificaciones.
2. Activar o desactivar los recordatorios de entrenamiento y dieta.
3. Seleccionar el horario deseado.
4. Guardar la configuración.

7.3 Cerrar sesión

1. Acceder al perfil.
2. Pulsar Cerrar sesión.

Nota: Visualizar las Figuras 35, 36 y 37.

9.2 Manual de programador

En este apartado se describen los principales aspectos técnicos de FitVive con el objetivo de facilitar su comprensión y su mantenimiento.

Requisitos previos del entorno

Antes de comenzar, es necesario disponer del siguiente entorno de desarrollo:

- Android Studio, utilizado como entorno de desarrollo utilizado para desarrollar y ejecutar el proyecto.
- JDK 11 (Java Development Kit), necesario para la compilación de la aplicación.
- Android SDK, instalado desde Android Studio, para poder ejecutar la aplicación en un dispositivo físico o en un emulador.
- Xcode, necesario para la compilación y ejecución en dispositivos iOS.
- Conexión a internet, necesaria para utilizar las APIs externas empleadas por la aplicación, como Gemini, Wger y Open Food Facts, así como para la sincronización con Firebase.

Estructura del proyecto

FitVive está desarrollado mediante Kotlin Multiplatform, por lo que el proyecto se divide en diferentes carpetas.

Carpeta	Descripción
commonMain	Contiene la mayor parte de la aplicación, incluyendo pantallas, la lógica de negocio, los repositorios, la base de datos y los servicios de red.
androidMain	Contiene las implementaciones específicas de Android, como la sincronización con Firestore, las notificaciones y la actividad principal.
iosMain	Incluye funciones específicas para iOS y la integración de SwiftUI.
commonTest	Contiene las pruebas compartidas del proyecto.
iosApp	Contiene el proyecto de iOS necesario para ejecutar la aplicación.

Tabla 14. Estructura del proyecto. Fuente: Elaboración propia.

Tecnologías utilizadas

Tecnología	Función
Kotlin Multiplatform	Desarrollo multiplataforma.
Compose Multiplatform	Interfaz de usuario.
SQLDelight	Base de datos local.
Firebase Firestore	Sincronización de datos.
Ktor	Comunicación con APIs.
Gemini API	Asistente basado en IA.
Wger API	Información sobre ejercicios.
Open Food Facts API	Información nutricional.

Tabla 15. Tecnologías utilizadas. Fuente: Elaboración propia.

Organización de la aplicación

FitVive organiza la mayor parte de su funcionamiento en el módulo `commonMain`, donde se encuentran las pantallas, la lógica de negocio y los repositorios. La navegación entre pantallas se gestiona desde `App.kt`, que también se encarga de gestionar el estado de la aplicación y la comunicación con los distintos servicios.

La información del usuario se almacena mediante `SQLDelight`, mientras que la comunicación con los servicios externos se realiza utilizando `Ktor`.

Ejecución del proyecto

Android:

1. Abrir el proyecto en Android Studio [19].
2. Esperar a que finalice la sincronización de Gradle.
3. Seleccionar un dispositivo o emulador.
4. Ejecutar la aplicación mediante `Run`.

iOS:

1. Abrir el proyecto desde Xcode utilizando el proyecto `iosApp.xcodeproj`.
2. Esperar a que finalice la carga del proyecto y de sus dependencias.
3. Seleccionar un simulador o un dispositivo iOS compatible.
4. Pulsar el botón `Run` para compilar y ejecutar la aplicación.

Arquitectura del proyecto

FitVive utiliza una arquitectura basada en Kotlin Multiplatform y Compose Multiplatform, donde la mayor parte de la lógica de la aplicación se encuentra en el código compartido (`commonMain`). La aplicación se organiza por pantallas y servicios, utilizando repositorios para el acceso a los datos y el patrón `expect/actual` para implementar las funcionalidades específicas de Android e iOS.

9.3 Configuración de APIs

FitVive utiliza diferentes APIs externas para ofrecer algunas de sus funcionalidades. Antes de ejecutar la aplicación es necesario configurar correctamente estos servicios.

Gemini API

1. Acceder a Google AI Studio.
2. Crear una clave de API (API Key).
3. Copiar la clave generada y añadirla en el archivo `AsistenteApiService.kt`.
4. Guardar los cambios y volver a ejecutar la aplicación.

Firebase Firestore

1. Crear un proyecto en Firebase.
2. Activar Cloud Firestore.
3. Descargar el archivo `google-services.json`.
4. Copiar el archivo en la carpeta `composeApp`.
5. Sincronizar el proyecto con Gradle.

Wger API

No requiere realizar ninguna configuración adicional, ya que la aplicación utiliza directamente la API publica de Wger.

Open Food Facts API

Tampoco requiere ninguna configuración adicional. La aplicación realiza las consultas utilizando la API pública de Open Food Facts.

9.4 Código relevante

En este apartado se muestran algunos fragmentos del código desarrollado durante el proyecto. Estos ejemplos corresponden a algunas de las funcionalidades más importantes de FitVive.

1. Registro de usuarios

Este fragmento muestra el proceso de registro de un nuevo usuario. Antes de almacenar la información, se comprueba que el correo electrónico no exista previamente en la base de datos

```
suspend fun registrarUsuarioSiNoExiste(nombre: String, email: String, password: String): Boolean =
    withContext(Dispatchers.Default) {
        val emailNormalizado = email.trim().lowercase()
        database.transactionWithResult {
            if (queries.existeUsuarioConEmail(emailNormalizado).executeAsOne()) {
                false
            } else {
                queries.guardarNuevoUsuario(
                    nombre = nombre.trim(),
                    email = emailNormalizado,
                    password = password,
                    sexo = "",
                    peso = "",
                    altura = "",
                    fechaNacimiento = "",
                    objetivo = "",
                    diasEntrenamiento = 0,
                    grupoMuscularFoco = ""
                )
                true
            }
        }
    }
}
```

Figura 38. Código correspondiente al registro de un usuario. Fuente: Elaboración propia.

2. Entrenamientos

Este fragmento muestra cómo el usuario accede al detalle de un entrenamiento y cómo, al finalizar la sesión, se guardan los datos del entrenamiento realizado.

```
"entrenamientos" -> EntrenamientosScreen(  
    diasEntrenamiento = diasEntrenamiento,  
    objetivo = objetivoSeleccionado,  
    grupoMuscularFoco = grupoMuscularFoco,  
    onDiaClick = { dia ->  
        diaSeleccionado = dia  
        currentScreen = "detalleEntrenamiento"  
    },  
    onInicioClick = { currentScreen = "home" },  
    onDietaClick = { abrirDieta() },  
    onPerfilClick = { currentScreen = "perfil" }  
)  
  
"detalleEntrenamiento" -> DetalleEntrenamientoScreen(  
    dia = diaSeleccionado,  
    totalDias = diasEntrenamiento,  
    objetivo = objetivoSeleccionado,  
    grupoMuscularFoco = grupoMuscularFoco,  
    apiRepository = apiRepository,  
    httpClient = httpClient,  
    onFinalizarClick = { grupoMuscular, registros ->  
        val fecha = obtenerFechaActual()  
        scope.launch {  
            repository.guardarEntrenamiento(fecha, diaSeleccionado, grupoMuscular)  
            syncer.syncEntrenamiento(emailUsuario, fecha, diaSeleccionado, grupoMuscular)  
            syncer.syncSeriesEjercicio(emailUsuario, fecha, diaSeleccionado, grupoMuscular, registros)  
        }  
        currentScreen = "entrenamientos"  
    }  
)
```

Figura 39. Código correspondiente a la realización de un entrenamiento. Fuente: Elaboración propia.

3. Dieta

Este fragmento muestra cómo la aplicación consulta la API de Open Food Facts para obtener información nutricional de un alimento y adaptarla a las calorías asignadas.

```
suspend fun getComida(objetivo: String, tipo: String, caloriasObjetivo: Int): ComidaAPI? {
    val query = queryParaTipo(tipo, objetivo)

    val response: OFFResponse = client.get( urlString = "https://world.openfoodfacts.org/cgi/search.pl" ) {
        parameter("search_terms", query)
        parameter("action", "process")
        parameter("json", 1)
        parameter("page_size", 5)
        parameter("fields", "product_name,nutriments")
    }.body()

    val producto = response.products.firstOrNull {
        it.nombre.isNotBlank() && it.nutriments.energiaKcal100g > 10.0
    } ?: return null

    // Calcular porción (g) para alcanzar el objetivo calórico
    val gramos = caloriasObjetivo / (producto.nutriments.energiaKcal100g / 100.0)
    val factor = gramos / 100.0

    return ComidaAPI(
        nombre = "${producto.nombre} (${gramos.roundToInt()}g)",
        calorias = caloriasObjetivo,
        proteinas = (producto.nutriments.proteinas100g * factor).roundToInt(),
        carbohidratos = (producto.nutriments.carbohidratos100g * factor).roundToInt()
    )
}
```

Figura 40. Código correspondiente a la obtención de información nutricional mediante Open Food Facts. Fuente: Elaboración propia.

4. Asistente IA

Este fragmento muestra cómo se prepara la consulta del usuario y se envía a Gemini API para generar una respuesta relacionada con entrenamiento o alimentación.

```
1 Usage
suspend fun enviarMensaje(
    historialPrevio: List<Pair<String, String>>,
    nuevoMensaje: String,
    contextoUsuario: String
): String {
    val contents = historialPrevio.map { (rol, texto) ->
        ChatContent(role = rol, parts = listOf(ChatPart(text = texto)))
    } + listOf(ChatContent(role = "user", parts = listOf(ChatPart(text = nuevoMensaje))))

    val systemPrompt = buildString {
        append("Eres un asistente fitness personal amigable integrado en la app FitVive. ")
        if (contextoUsuario.isNotBlank()) append(contextoUsuario)
        append("Responde siempre en español, de forma concisa (máximo 3-4 frases), práctica y motivadora. ")
        append("Solo responde sobre ejercicio, nutrición, hábitos saludables y fitness. ")
        append("Si te preguntan algo fuera de ese ámbito, redirige amablemente al tema fitness.")
    }

    val request = ChatRequest(
        systemInstruction = ChatSystemInstruction(
            parts = listOf(ChatPart(text = systemPrompt))
        ),
        contents = contents
    )

    val response: ChatResponse = client.post( urlString = API_URL ) {
        contentType(ContentType.Application.Json)
        setBody(request)
    }.body()

    return response.candidates.firstOrNull()?.content?.parts?.firstOrNull()?.text
        ?: "No pude obtener una respuesta. Inténtalo de nuevo."
}
```

Figura 41. Código correspondiente al envío de consultas al asistente basado en inteligencia artificial. Fuente: Elaboración propia.

5. Notificaciones

Este fragmento muestra cómo la aplicación programa una notificación diaria en Android utilizando la hora configurada por el usuario.

```
4 Usages
private fun scheduleAlarm(requestCode: Int, hora: Int, minuto: Int, receiverClass: Class<*>) {
    val intent = Intent(packageContext = context, cls = receiverClass)
    val pendingIntent = PendingIntent.getBroadcast(
        context, requestCode, intent,
        flags = PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
    )
    val cal = Calendar.getInstance().apply {
        set(Calendar.HOUR_OF_DAY, hora)
        set(Calendar.MINUTE, minuto)
        set(Calendar.SECOND, 0)
        set(Calendar.MILLISECOND, 0)
        if (timeInMillis <= System.currentTimeMillis()) add(p0 = Calendar.DAY_OF_YEAR, p1 = 1)
    }
    alarmManager.setInexactRepeating(
        type = AlarmManager.RTC_WAKEUP,
        triggerAtMillis = cal.timeInMillis,
        intervalMillis = AlarmManager.INTERVAL_DAY,
        operation = pendingIntent
    )
}
```

Figura 42. Código correspondiente a la programación de notificaciones diarias. Fuente: Elaboración propia.